มหาวิทยาลัยอัสสัมชัญ
ASSUMPTION UNIVERSITY
of THAILAND

VINCENT MARY
SCHOOL of ENGINEERING, SCIENCE and
TECHNOLOGY

Senior Project Report

Adaptive Traffic Signal Control System

A Camera-Based, Algorithm-Driven Approach for T-Intersections:
Case Study of the ABAC University Entrance, Bang Na, Samut Prakan

---

Ratchanon Puwapattarapokin (6610909),
Warachai Aranchot (6610996),
Badin Bangsen (6611108)

Asst. Prof. Dr. Darun Kesrarat (Advisor)

CSX3010 Senior Project 1 (1/2026)

Senior Project Approval

Project title: Adaptive Traffic Signal Control System

Academic Year: 1/2026

Authors: Ratchanon Puwapattarapokin (6610909)
Warachai Aranchot (6610996)
Badin Bangsen (6611108)

Project Advisor: Asst. Prof. Dr. Darun Kesrarat

The Senior Project committee's cooperation between the Department of Computer Science and Information Technology, Vincent Mary School of Engineering, Science and Technology, Assumption University had approved this Senior Project. The Senior Project in partial fulfilment of the requirement for the degree of Bachelor of Science in Computer Science and Information Technology.

Approval Committee:

……

(Asst. Prof. Dr. Darun Kesrarat)

Project Advisor

……

(Asst. Prof. Dr. Anilkumar Kothalil Gopalakrishnan)

Committee Member

……

(Dr. Phyo Min Tun)

Committee Member

Abstract

The signal at the entrance to Assumption University (ABAC), on the Bang Na-Trat service road, runs a fixed-time plan with a 107 s cycle. We measured it and found it hands out green almost the opposite way to demand. Bx, the right turn into the university, gets 20 s of green but needs 28.6 s (volume-to-capacity ratio, v/c, 1.51), so its queue reaches the four-way junction 78 m behind it in about 5.7 minutes. Through movement A needs only 12.3 s of its 40 s (v/c 0.25). Moving ten seconds of green from A to Bx would clear the queue. We trained a YOLOv8s detector on 6,585 hand-labelled images from about 6.6 hours of drone footage, and its counts came within 1.7% of a hand count.

Our controller, SODA, reads one vehicle count per signal head every second, serves the queue that formed first, and sizes each green from the count, using the measured 2.31 s saturation headway and 2.95 s startup lost time. In a Unity simulation calibrated to the measured discharge rate, both controllers ran on identical traffic in 108 pairs of runs. Mean wait per vehicle changed by −12.9%, worst-case wait by −41.6% and wait on Bx by −53.9%, and SODA moved green from A to Bx without being told to. The gain is largest in the morning and midday scenarios. In the evening peak, when every approach is full at once, SODA's average wait was 6.8% higher than the fixed-time plan's (p = 0.024). These are simulation results: SODA has not run a real signal, simulated queue lengths have not been compared with observed ones, and the work covers dry daylight only.

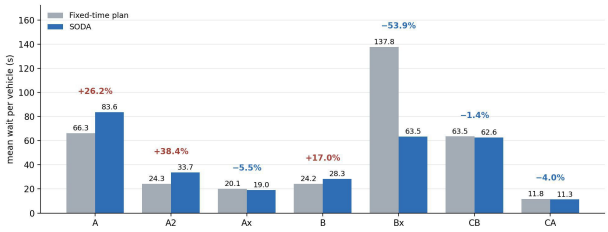
Keywords: adaptive traffic signal control; computer vision; YOLOv8; vehicle counting; traffic simulation; Unity

Results at a glance

The problem. The fixed-time signal gives the right turn into the university, Bx, 20 s of green when it needs 28.6 s (v/c 1.51). Through movement A gets 40 s and needs 12.3 s (v/c 0.25). Bx's queue reaches the junction 78 m behind it in about 5.7 minutes.



Bx queued seven deep while the B through lane beside it is empty (Section 5.4).



Mean wait per vehicle by movement in simulation, fixed-time plan against SODA (Section 6.16).

The fix within the existing plan. Moving green from A to Bx (Section 5.4):

Green moved from A to Bx	Bx v/c	Outcome
5 s	1.17	queue still grows
10 s	**0.95**	**queue clears**
12 s	0.88	clears comfortably

What SODA did in simulation (108 paired runs, Section 6.17):

KEY RESULT

Measure	Fixed-time plan	SODA	Change
Mean wait per vehicle	44.15 s	38.44 s	**−12.9%**
Worst-case wait	204.2 s	119.3 s	**−41.6%**
Wait on Bx	137.8 s	63.5 s	**−53.9%**

SODA found the A-to-Bx reallocation on its own, from vehicle counts alone. In the evening peak, when every approach is full, its average wait was 6.8% higher than the fixed-time plan's (p = 0.024) (Section 6.15.2).

Contents

1 · Introduction **8**

1.1 Background 8

1.2 Problem statement 8

1.3 Objectives 9

1.4 Scope and limits 9

1.5 How this report is organised 10

2 · Site description and field measurement **10**

2.1 Site geometry and movement definitions 11

2.2 The measured signal plan 12

2.3 Operating mode: the flashing-amber finding 13

2.4 Locally measured traffic parameters 14

2.5 Interview evidence 15

3 · Module A: vehicle detection **16**

3.1 Data collection 16

3.2 Annotation 17

3.3 Splitting the dataset by session 18

3.4 Preprocessing and augmentation 18

3.5 Training 19

3.6 Detection results 21

3.7 Error analysis 24

4 · Vehicle counting and validation **26**

4.1 Counter design 26

4.2 Validation against hand counts 28

4.3 The one consistent error 29

4.4 Calibration across sessions 29

4.5 The heavy class: a mismatch of definitions, not a detection failure 30

4.6 Platform drift 30

4.7 ROI depth: a known over-count we did not correct 31

5 · Field findings: demand, capacity and the central finding **32**

5.1 Method 32

5.2 Demand concentrates on the movement the signal does not control 33

5.3 The morning peak is later than the collection guide assumed 34

5.4 The central result: Bx is starved while A is over-served 35

5.5 The exit movements: Lane C 36

5.6 The shared lane: how blocking occurs 38
5.7 The same failure appears at both ends of the junction 39
5.8 Every signalised movement, measured 40
5.9 Visual confirmation: the result in a single frame 40

6 · Modules B and C: adaptive control and simulation 42
6.1 Controller architecture 42
6.2 Two separate rules: which head goes, and for how long 43
6.3 Ending a green early: the calculated green is a ceiling 45
6.4 Decision sequence 46
6.5 Conflict matrix and which movements can go together 48
6.6 Simulation environment 48
6.7 Test scenarios 50
6.8 Statistical analysis 51
6.9 Interface between the modules: measured arrival rates 52
6.10 Validation of the simulation against observed behaviour 53
6.11 The experiment 54
6.12 Main result: waiting time per vehicle 56
6.13 Worst-case waiting time 57
6.14 Throughput 58
6.15 Results by scenario 58
6.16 Results by movement: did the controller find the fix? 60
6.17 Summary of the simulation result 62

7 · Limitations 63
7.1 What the field data does not cover 63
7.2 Heavy vehicles and platform drift 64
7.3 What the simulation leaves out 64
7.4 What the controller cannot reach 65
7.5 Validation still to be done 66

8 · Conclusions and future work 66
8.1 What we found 66
8.2 Recommendation 67
8.3 Future work 67

9 · References 68

Appendix A · Arrival rates 68

Appendix B · Dataset settings 69

Appendix C · Detector calibration 71

Appendix D · Controller implementation 73

Appendix E · Training environment 76

Appendix F · Development log 78

Appendix G · Platform drift measurement 79

Movement key

A	eastbound through
A2	eastbound through, kerb lane shared with Ax
Ax	left turn into the university, no signal
B	westbound through
Bx	right turn into the university
CB	exit, turning west
CA	exit, turning east, no signal

The colours match the site map in Figure 2.1.

1 · Introduction

> **IN SHORT**
> The fixed-time signal at the ABAC entrance gives green almost the opposite way to demand: Bx, the right turn into the university, is overloaded (v/c 1.51), while through movement A gets more than three times the green it needs (v/c 0.25). We measured the junction, built a drone-camera vehicle counter and an adaptive controller (SODA), and tested SODA against the current plan in simulation.

1.1 Background

A fixed-time plan repeats the same timings whatever the traffic, so it hands out green by schedule rather than by demand (Webster, 1958). Adaptive systems such as SCATS (Lowrie, 1990) and SCOOT (Hunt et al., 1982) fix this by measuring demand, and Google and DeepMind now tune signals across whole cities, including Bangkok, from GPS data (Google Thailand / DeepMind, 2023). None fits this site: SCATS and SCOOT need loop detectors cut into the road, which a university cannot do, and city-wide GPS data is too coarse to see one gate. This project fills that gap: **an adaptive controller for one isolated junction that senses demand with a camera on an existing pole.**

The junction is not a typical main-road one. Soi ABAC dead-ends about 9 km beyond it, so its traffic is mostly university access, and the volumes in Chapter 5 are far lower than the name "Bang Na-Trat" suggests.

1.2 Problem statement

The project proposal said the signal "runs on a fixed 30-second cycle per phase". Fieldwork showed that was wrong.

Table 1.1 · The cycle length as stated before fieldwork and as measured

Source	Cycle length	Basis
Project proposal	30 s per phase	verbal, second-hand
Simulation's original model	120 s	assumed
Derived from a guard's verbal report	~115 s	interview
Measured	**107.0 s**	three independent methods

We timed the plan three ways (stopwatch, the signal's countdown display, and lamp colour read from video), and they agree within 3% (Section 2.2). This matters: a comparison against an invented 120 s plan would have meant nothing.

> At the ABAC entrance, the fixed signal plan gives green time almost the opposite way to demand. Both movements serving the university are at or beyond capacity (volume-to-capacity ratio, v/c, above 1.0), while both through movements hold more than three times the green they need. One queue backs up into a neighbouring junction within about six minutes at peak.

1.3 Objectives

The system has three modules with defined interfaces, so each can be built and tested on its own and joined through a documented data format.

Table 1.2 · Modules, owners and the data passed between them

Module	Responsibility	Input	Output
A · Vehicle detection	Ratchanon P. · Warachai A.	drone video	per-movement vehicle counts (JSON)
B · Adaptive control	Ratchanon P. · Warachai A. · Badin B.	counts	phase order and green durations
C · Simulation and evaluation	Badin B.	control decisions	delay, queue length, throughput

The objectives are:

1. To measure, rather than assume, the demand and signal timing at the study site.
2. To build a camera-based vehicle counter accurate enough that its output can be used directly as simulation input.
3. To design an adaptive control algorithm that sizes green time from measured queues and serves movements in the order their queues formed.
4. To evaluate that algorithm against the measured fixed-time plan in simulation, and to compare simulated behaviour against behaviour observed on site.

1.4 Scope and limits

Five boundaries apply to the results.

The drone is a measuring tool, not the sensor. It was the available way to get an overhead view of a junction the project does not own. A deployed system assumes a **fixed mast- or pole-mounted camera** at a similar height and angle. We say this early because a reader who assumes the drone must fly all day would fairly conclude the system is impractical.

One approach at a time. The drone films one approach (one road arm with all its lanes) per flight, so the whole junction never appears in one frame. That is enough for measuring arrival rates offline, which is all this project needs, and is another reason to use a fixed camera in a deployment.

Dry daylight only. The dataset contains no wet-weather images, for the reasons in Chapter 7, and night operation is out of scope.

Obstruction beyond the junction is out of scope. The controller sizes green from the demand waiting at the stop line and cannot tell whether the road past the junction can take the vehicles it releases. An incident or a standing queue further down the road would have it giving green to a movement that cannot move. Any demand-responsive signal has this weakness, and no sensor in this project sees past the junction. This is the opposite direction to the spillback in Section 7.3, where this junction's own queue reaches a neighbouring one, which we measured and report.

The counter counts arrivals, not departures. A vehicle is counted once, when its tracked centre first enters a region and it has clearly moved (Section 4.1). Nothing confirms that a vehicle released on green actually cleared the junction; that would need a second detection zone past the stop line and a way to match vehicles between zones. Chapter 5 does not need it, because it uses the saturation headway measured directly from footage (Section 2.4) rather than working capacity out from throughput. **Wherever this report gives a number of vehicles discharged, it is computed from the measured headway, not observed vehicle by vehicle.**

1.5 How this report is organised

Chapter 2 describes the site and our field measurements, Chapter 3 the vehicle detector, and Chapter 4 the counter and its check against hand counts. Chapter 5 sets out the field findings: demand, capacity and the central finding. Chapter 6 describes our controller, SODA (Sections 6.1 to 6.5), the simulation (Sections 6.6 to 6.10) and the simulation results (Sections 6.11 to 6.17). Chapter 7 sets out the limitations, Chapter 8 the conclusions and future work, and Chapter 9 the references. Appendices A to G hold supporting detail.

2 · Site description and field measurement

> **IN SHORT**
> The signal runs a fixed 107 s cycle and controls the junction for only 7 of every 24 hours; the rest of the day it flashes amber. Queues leave at 2.31 s per vehicle with 2.95 s lost at each start, slower than the textbook defaults of 2.00 s.

These measurements come from site visits in July and August 2026, and the rest of the report rests on them. Where a measurement replaced an assumption, we show both.

2.1 Site geometry and movement definitions

The ABAC entrance is a true T-intersection. The Bang Na-Trat service road runs east-west as the main road, and the university entrance forms the stem of the T, to the south. The report uses the same six movements throughout.

Table 2.1 · The six movements and how each is controlled

Movement	Description	Control
A	eastbound through	signalised
Ax	eastbound **left turn** into ABAC	**unsignalised, free flow**
B	westbound through	signalised
Bx	westbound **right turn** into ABAC	signalised
CB	exits ABAC, turns west onto the main road	signalised
CA	exits ABAC, turns east onto the main road	**unsignalised, free flow**

A signalised movement has a traffic light; on a free-flow movement drivers go when the way is clear. A2 is straight-on traffic in the kerb lane of approach A, which it shares with the Ax left turn.

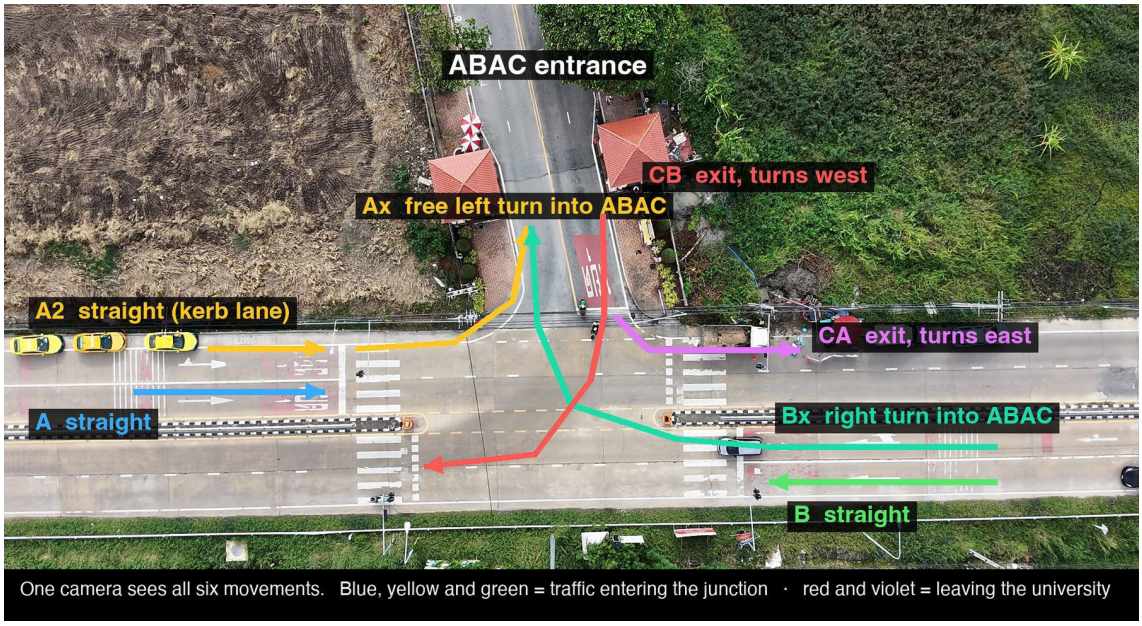


Figure 2.1. The junction seen from the overhead survey flight, with the six movements used throughout this report. A and B are the through movements, Ax and Bx turn into the university, and CB and CA leave it. A2 is straight-on traffic in the kerb lane that A shares with the Ax left turn.

Ax and CA have no signal. Both are left turns that cross no other stream, so normal practice would not signalise them. Chapter 5 shows that both are still blocked in practice, and that this blocking is how the junction typically fails.

Queue storage

Queue storage is the length of road a queue can fill before it reaches something else, and only one approach is limited by it. The A and C approaches are long enough that their queues never run out of road, so signal capacity sets their limit. The B approach ends at a neighbouring four-way junction about 78 m upstream. Once the queue fills that distance it spills into the second junction and delays traffic that never meant to use ours. Two measurements agree on the distance: site observation gives about 78 m, and the longest Bx queue in the footage, 12 vehicles at about 6.5 m each, is also about 78 m.

So the longest queue we saw uses all the road there is. Bx already runs at the edge of spillback in normal peak traffic, and any growth in demand would turn a local delay into a delay on the wider network.

2.2 The measured signal plan

We timed the signal on site on 23 August 2026 by three independent methods. A phone stopwatch recorded each change of light. The countdown display, which only the C approach's signal head has, was read directly and used to work out the other heads. And an automated analysis sampled the lamp colour at 2 Hz from a 4 minute 52 second ground-level recording, which gave two complete cycles before the camera drifted.

A signal head is one physical traffic light. The junction has four (A, B, Bx and CB), and A's head also controls A2.

Table 2.2 · Signal timings measured on site, per head

Head	Green	Amber	Red	Implied cycle
A	40 s	3 s	63 s	106 s
B	65 s	3 s	35 s	103 s
Bx	20 s	3 s	84 s	107 s
CB	31 s	3 s	75 s	109 s

The four timings give 103-110 s, with a mean of **107.0 s**, which matches the automated measurement of the CB head exactly. Timing by hand is accurate to ±3-4 s, so this is as close as the method allows. The plan runs in this order:

```
t =    0 – 40    A green  +  B green
t =   40 – 43    A amber      (B remains green)
t =   43 – 45    all-red
t =   45 – 65    Bx green  +  B green
t =   65 – 68    B amber  +  Bx amber
t =   68 – 70    all-red
t =   70 – 101   CB green
t =  101 – 104   CB amber
t =  104 – 106   all-red
```

We call the fixed plan's phases stages. All-red is a short period when every head shows red so the junction clears before the next green.

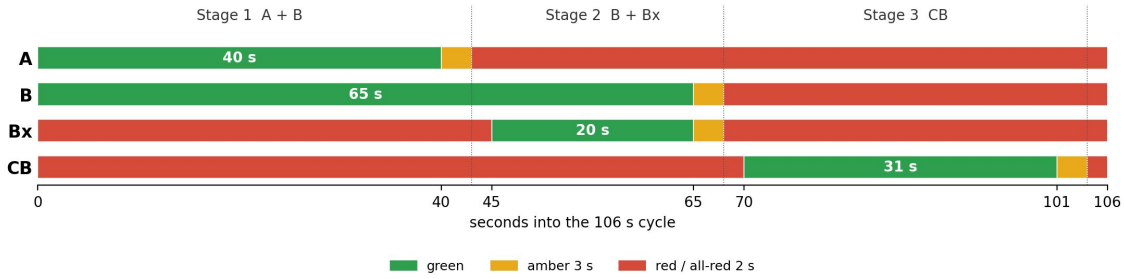


Figure 2.2. The measured fixed-time plan over one 106 s cycle. B stays green through stages 1 and 2, so Bx joins B 45 s into the cycle instead of having a stage of its own. Every change of stage costs 3 s of amber and 2 s of all-red.

Two features matter for the rest of the report. First, Bx has no stage of its own: it joins B about 45 seconds into the cycle, part-way through B's green. That matches the conflict matrix used in the control design, the table of which movements cross and so must never be green together, in which A runs with B, B runs with Bx, and CB runs alone. Second, the signal is fixed-time, not vehicle-actuated. Two red periods in a row on the CB head both measured 73.5 s, where an actuated signal would have varied them with demand. So Chapter 6 compares SODA against a plan that does not respond to traffic at all.

2.3 Operating mode: the flashing-amber finding

Security staff told us the signal's daily schedule, and we checked it against every recorded session.

Table 2.3 · Daily schedule: normal red/green or flashing amber

Period	Regime

Period	Regime
05:00-09:00	normal red / green
09:00-15:00	**flashing amber**
15:00-18:00	normal red / green
18:00-05:00	**flashing amber**

In flashing amber no movement has an enforced right of way, so the signal controls the junction for 7 hours in every 24, about 29% of the day, and drivers sort it out among themselves the rest of the time. The schedule matched all four sessions we had recorded before we knew about it: the two morning sessions fell under normal signalling and the two midday sessions under flashing amber, as predicted.

Why this motivates the project

Flashing amber probably exists because a fixed-time plan wastes capacity when demand is low: a full cycle at 03:00 delays vehicles that have nothing to wait for. The operator's answer is to switch off the right of way altogether, trading safety for efficiency at a junction where accidents have already happened. An adaptive controller removes that trade-off. It serves only the demand it measures, so light traffic costs little delay and there is no efficiency reason to fall back to flashing amber.

The guard we interviewed raised this himself. Asked whether he had anything to add, he said, unprompted, that he **dislikes the flashing-amber mode and would prefer full red/green signalling to run 24 hours**, for safety. The person who runs the junction day to day asked for what this project proposes, and the measurements explain why the current signal cannot provide it.

2.4 Locally measured traffic parameters

Capacity analysis needs a saturation flow, the most vehicles one lane can pass in an hour of continuous green. The Highway Capacity Manual (HCM) gives default values for when local measurements are missing (Transportation Research Board, 2022). We measured it instead, from footage of five green phases that began with standing queues, which gave 39 saturation-headway samples.

Table 2.4 · Traffic parameters measured here against the HCM defaults

Parameter	Measured here	HCM 7th ed. default
Saturation headway	**2.31 s/veh** (sd 0.81)	2.00
Saturation flow	**1558 veh/hr/lane**	1800
Startup lost time	**2.95 s per cycle**	2.00

Startup lost time here is about 1.5 times the HCM default. That is believable at this junction, because motorcycles move to the front of the queue during red and must clear the stop line before the cars behind them can move. We treat it as a finding, not a detail, because Section 5.4 shows it changes the project's main result.



Figure 2.3. Fieldwork. Left: the drone taking off beside the gate. Right: the live view on the controller during a counting flight over approach B.

2.5 Interview evidence

We interviewed a security guard stationed at the front gate; the interview was audio-recorded and transcribed. Five of his points bear on the analysis.



Figure 2.4. Ratchanon P. interviewing the security guard at the front gate, who described the signal schedule, the daily Lane C closure and how guards wave traffic through.

Weekday volumes differ a lot. He described Tuesday to Thursday as full-strength days, Monday as about 70% and Friday as 40-50%. We use this only to describe sessions, never as a correction factor (Chapter 7).

The Lane C exit closes every day between 08:30 and 09:00, for 15 to 30 minutes, because of congestion and past accidents, which pushes outbound traffic onto another route. Section 5.3 shows this window falls exactly on the arrival peak we measured separately.

Taxis wait across three lanes on the A approach, including the emergency lane, until they are called in, as university policy requires. They are parked, not traffic, and Section 4.1 describes how the counter leaves them out.

Exit capacity changes with the time of day. Another way out opens between 15:30 and 17:00, which a simulation with a fixed road layout does not show.

People, not the signal, manage the junction. The guard waves left-turning vehicles into the university himself, because straight-on vehicles stop and block the turn lane, and he estimated he does this 40-60% of the time. Section 5.6 measures the mechanism behind it. So **any "before" baseline at this junction measures the guards' performance as much as the signal's.**

3 · Module A: vehicle detection

> **IN SHORT**
> We trained a YOLOv8s detector on 6,585 hand-labelled drone images of the junction. On four sessions it never saw in training it scores 0.883 on mAP@50, the standard detection score (car 0.972, motorcycle 0.911, heavy 0.766; Section 3.6).

3.1 Data collection

We filmed the junction by drone on three approaches at three times of day between July and August 2026, about 6.6 hours of video in 20 session folders. The counting flights hovered about 6 m above the road; a separate survey flight at 90 m captured the layout of the junction and the roads around it.

All footage is **1920 × 1080 at 29.97 frames per second (fps)**, HEVC-encoded. We checked this in every collection folder rather than assuming it, because the resolution argument in Section 3.4 depends on it.



Figure 3.1. One frame from each approach, taken from the counting flights at about 6 m. The same three viewpoints supply the training data, the calibration sessions and the demonstration footage.

A battery lasts 20-25 minutes, so each session is two or more video files. We kept each flight as its own session folder, for the reason in Section 3.3.

3.2 Annotation

We extracted a frame every 3.5 seconds and two annotators labelled them in Roboflow, an online labelling tool, drawing a box around each vehicle and naming its class. The final dataset has **6,585 annotated images containing 57,260 labelled instances** (one instance is one boxed vehicle in one image).

We cut the classes from five to three

The first dataset version had five classes: car, motorcycle, truck, bus and special type. Its confusion matrix, a table of how often each true class is predicted as each class, showed that the largest error was trucks and cars mistaken for each other.

The cause is resolution, not careless labelling. From 6 m up, the open bed that tells a pickup from a saloon car covers a few pixels; annotators disagree on these cases, and more labelling cannot settle a difference the camera does not record. So we merged the classes into the passenger-car-unit (PCU) groups the capacity analysis uses.

Table 3.1 · The three detection classes, their PCU weights and annotation counts

Class	PCU weight	Instances	Share
car	1.0	44,622	77.9%
motorcycle	0.33	11,982	20.9%
heavy	2.0	656	1.1%

The analysis never needed to tell a truck from a bus; it needed PCU-weighted flow, so we matched the model's output to that. Heavy vehicles are still rare at 1.1% of instances, though that is four times the 160 in the first version. Section 4.4 shows the effect on counting each session, and Section 4.5 shows that the small sample is not why the class cannot be reported on its own. Appendix B gives the full dataset settings.

3.3 Splitting the dataset by session

The training set is what the model learns from, the validation set checks progress and picks the best point in training, and the test set is held back for the final score. How we made this split was the most important decision in the dataset.

Frames 3.5 seconds apart from a hovering drone are nearly identical: the same background, the same parked vehicles, often the same moving vehicles a few metres on. A random split would put near-copies in both training and validation, so the model would be checked on images it had in effect already seen. That is data leakage. Its mean average precision (mAP), the standard detector score from 0 to 1, would then measure memorisation instead of how it does on new footage. The usual form, mAP@50, counts a box as correct when it overlaps the true box by at least 50%.

We therefore split **by session folder**, with every folder entirely in one set.

Table 3.2 · Train, validation and test splits, by whole session

Split	Sessions	Images
Train	12	4,075 (62%)
Validation	4	1,406 (21%)
Test	4	1,104 (17%)

Every approach and time of day appears in training. The validation and test sessions are whole flights the model never saw, recorded on different days in different light. The test set is a little larger than the usual 70/20/10 because a session cannot be divided without bringing the leakage back; a clean 17% beats a contaminated 10%.

We assigned the split by hand across 20 folders with filename filters, so a script checked it before training. It maps every exported filename back to its session and stops if any session appears in more than one set. A filter that matched too much would not show in the Roboflow interface, and the final score would not reveal it either.

3.4 Preprocessing and augmentation

Roboflow turned each image the right way up and resized it to fit within 1280 × 1280 without changing its shape, so a 1920 × 1080 frame becomes 1280 × 720.

Our detector is YOLOv8 (Jocher et al., 2023), in its small version, YOLOv8s. Resizing to the more common 640 px would be a 3× downscale and shrink a distant motorcycle from about 20 pixels to about 7, below the size YOLOv8 detects reliably. Motorcycles are 21% of all instances, so that loss is not acceptable; 1280 is only a 1.5× downscale.

We chose "fit within" over "stretch to" because YOLO letterboxes images at inference (resizes without stretching and pads the rest), so a model trained on stretched images would meet differently shaped images in use.

Augmentation makes extra training images by changing real ones slightly. We used a 2× multiplier, which produced 8,150 training images from 4,075 source images.

Table 3.3 · Augmentations used, and why

Augmentation	Setting	Rationale
Flip	horizontal + vertical	free variety; an overhead view has no canonical orientation
Rotation	±5°	matches real between-flight heading variation on a stabilised gimbal
Brightness	±20%	real variance across morning, midday and evening sessions
Exposure	±10%	largely redundant with brightness; kept at low cost
Blur	0.2 px	negligible; daylight footage has a fast shutter
Noise	0.5%	reduced, see below

That is less than the first version used (3× multiplier, rotation ±15%, noise 1.01%). Augmentation stands in for real data, and the second dataset has 2.3 times as much real data, across three approaches, three times of day and two months of lighting. Noise was cut the most: when the target is a few pixels across, pixel noise risks teaching the model that speckle and distant motorcycles are the same thing. We left out shear, heavy distortion, large hue shifts, greyscale and auto-contrast, because those changes never happen at this site.

3.5 Training

We trained by transfer learning: start from a model already trained on other images, then continue training it on ours. There were four stages, each starting from the last:

```
COCO  →  VisDrone  →  ABAC v1  →  ABAC v2
```

Stage one is the standard YOLOv8s pretrained on COCO, a large public dataset of everyday photos, downloaded ready-trained from Ultralytics. Stage two fine-tunes it on VisDrone (Du et al., 2019), a public drone dataset, which adapts the backbone (the part that picks out image features) to overhead views and small objects. Stage three fine-tunes on the first ABAC dataset and stage four on the final one. All our training ran in Google Colab on an NVIDIA T4 GPU; Appendix E gives each stage's settings.

Table 3.4 · Settings for the final training stage

Parameter	Value
Architecture	YOLOv8s (11.1 M parameters, 28.6 GFLOPs)

Parameter	Value
Image size	1280
Batch size	8
Epochs	30
Optimiser	AdamW, lr 0.001429 (auto-selected)
Early stopping	patience 15

An epoch is one pass over all training images and batch size is how many are processed at once. Early stopping ends training if the validation score has not improved for 15 epochs. The optimiser and its learning rate (lr) were left at the library's automatic choice.

The architecture comes with the checkpoint, the saved weights file such as `best.pt`: loading the stage-three weights fixes it whatever the settings say. We kept YOLOv8s because it gave the counting accuracy in Section 4.2, and swapping architecture late in the project would have traded a checked result for an unchecked one. Because the class count fell from five to three, the detection head (the part that outputs boxes and classes) was partly rebuilt. Ultralytics matched the `car` and `motorcycle` rows by name and kept them, only the `heavy` row started from scratch, and all 355 backbone tensors transferred.

Training behaviour and the choice of epoch count

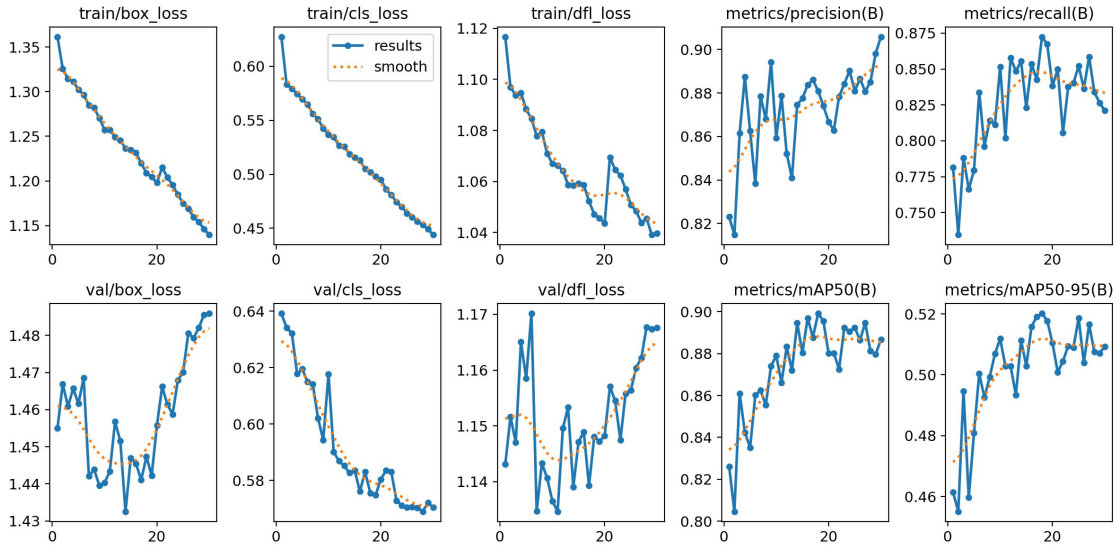


Figure 3.2. Training and validation curves for the final model over 30 epochs. Training losses fall steadily throughout. Validation mAP@50 levels off at about 0.88-0.90 from epoch 16, and the validation box loss begins to rise.

A loss measures how wrong the model is: box loss for where the box sits, classification loss for the class, and DFL loss as a second measure of box placement. All three training losses fall smoothly. The validation losses do not:

- `val/cls_loss` falls throughout, from 0.640 to 0.573
- `val/box_loss` reaches its lowest point near epoch 14, then **rises** from 1.44 to 1.485
- `val/dfl_loss` follows the same pattern
- `mAP@50` levels off from about epoch 16 at 0.88-0.90

That is the start of overfitting in box placement: the model keeps getting better at saying *what* an object is, but on unseen images slightly worse at placing the box. It matters little here, because the counter only checks whether a vehicle's centre crossed a polygon, not how tightly the box fits (Section 4.2). The checkpoint used throughout is `best.pt`, saved at peak validation mAP (about epoch 18). Thirty epochs were more than needed; 20 would have reached the same result in two-thirds of the time.

3.6 Detection results

The final stage trained for 30 epochs in 4.07 hours on the T4. We scored it on the **test** set, four complete sessions (`A_midday_2`, `B_midday_2`, `B_evening_aug14_3`, `C_midday_2`) that nothing in training saw, holding 1,104 images and 5,417 labelled instances.

Table 3.5 · Overall detection accuracy on unseen sessions

	mAP@50	mAP@50-95
Overall (test)	**0.883**	0.500
Overall (validation)	0.899	0.520

Validation and test agree to within 1.6%, so early stopping did not overfit to the validation set.

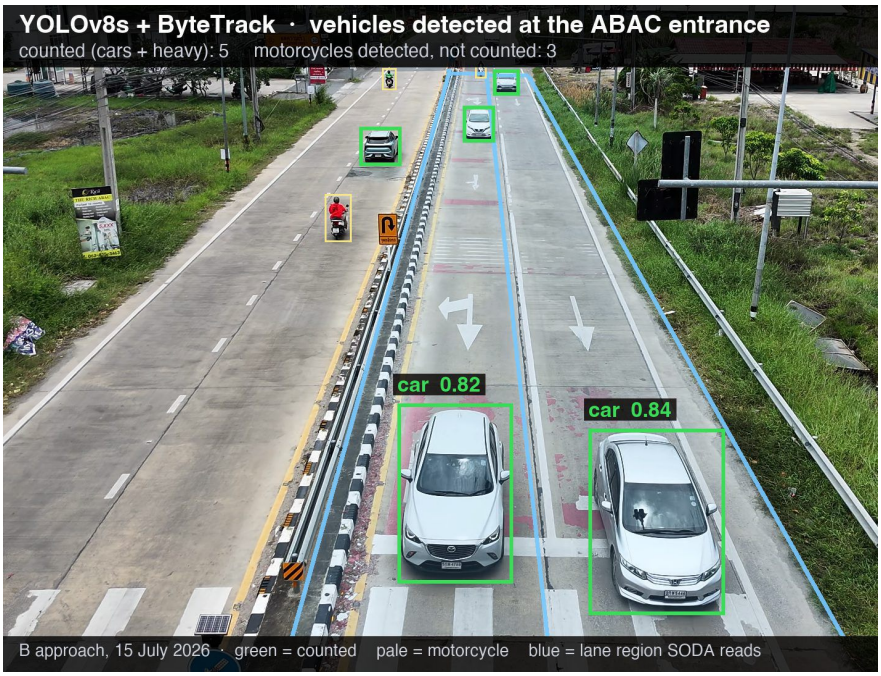


Figure 3.3. The final model on held-out footage from approach B. Cars and heavy vehicles inside the lane region are counted. Motorcycles are detected and drawn with thin boxes, but not counted.

Per class

We report each class so the rare heavy class is not hidden in the overall score. Precision is the share of drawn boxes that were real vehicles; recall is the share of vehicles present that were found.

Table 3.6 · Detection accuracy per class on the test set

Class	Test instances	mAP@50	mAP@50-95	Precision	Recall
car	4,066	**0.972**	0.608	0.937	0.923
motorcycle	1,281	**0.911**	0.438	0.929	0.854
heavy	70	0.766	0.456	0.771	0.714

Motorcycles improved the most. They are the smallest objects, 21% of instances and the worst affected by low evening light, yet precision of 0.929 with recall of 0.854 means the model finds them reliably without many false positives.

Heavy vehicles improved but remain the weakest class: recall of 0.714 means about 29% are missed. The test set holds only **70 heavy instances across 69 images**, so the true score could be well above or below this, and mAP does not decide whether the class is usable; Section 4.5 does.

The low motorcycle mAP@50-95 (0.438) does not matter here. mAP@50-95 averages the score over stricter and stricter overlap limits, measured as intersection over union (IoU), from 50% to 95%. On objects a few dozen pixels across, a box two pixels off

fails a strict IoU test while still marking the right vehicle in the right place, and Section 4.2 explains why that does not affect counting.

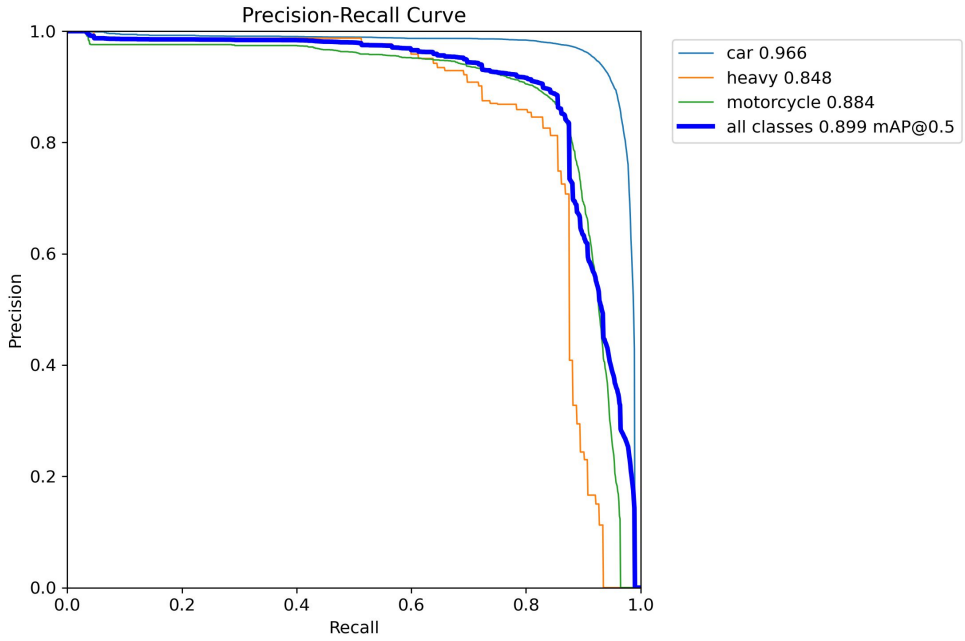


Figure 3.4. Precision-recall curves on the validation split, where mAP@50 is 0.899 overall. The figures in the table above are from the test split (0.883), which no part of training saw. Heavy is the weakest class on both.

Comparison with the first model

Table 3.7 · The first model against the final model

	v1	v2
mAP@50	0.764	**0.883**
Images	2,826	6,585
Classes	5	3
Sessions	8	20

The two scores were measured on different data. The v2 test includes Lane C and evening footage that did not exist when v1 was trained, so v2 scored higher on a harder test. The improvement is real and large, but 0.119 is not an exact measure of it.

Inference speed

On the T4, inference takes 10.4 ms per image at 1280 px, plus 0.5 ms preprocessing and 3.4 ms postprocessing: **about 70 frames per second end to end.** A pole-mounted camera system must process video at least as fast as it arrives, and a GPU of this class does so easily.

3.7 Error analysis

The confusion matrix shows that the **kind** of error changed between v1 and v2, not just the amount.

Validation set, 14,493 instances:

Table 3.8 · Confusion counts on the validation set

	Predicted car	Predicted heavy	Predicted moto	Missed
True car (11,236)	**10,695**	7	6	528
True heavy (152)	8	**128**	9	7
True motorcycle (3,105)	8	0	**2,767**	330
False positives	905	25	525	—

The truck/car confusion is gone

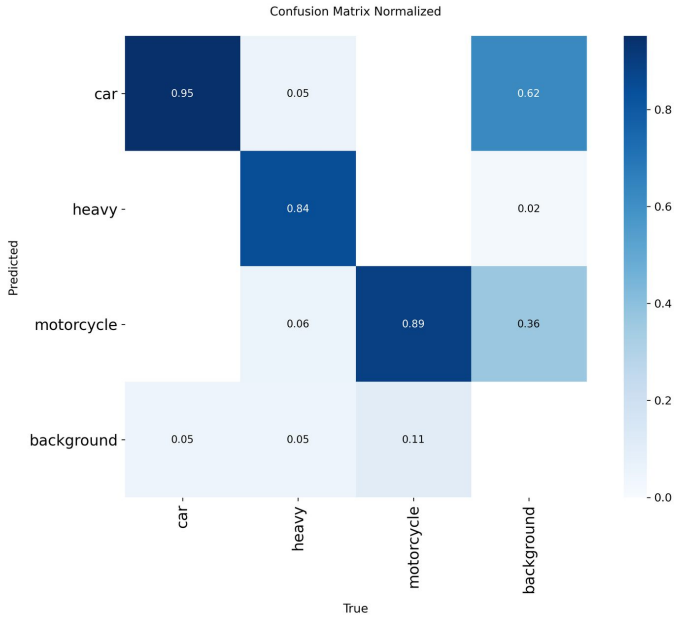


Figure 3.5. Normalised confusion matrix on the validation split. Each column is a true class. Almost nothing is confused between the three vehicle classes. The remaining errors are against background, which Chapter 4 shows the counter absorbs.

One vehicle class predicted as another now accounts for 38 instances out of 14,493, or 0.26%. That was v1's main failure and the reason for dropping the five-class taxonomy. A heavy vehicle is now classified correctly 84.2% of the time, as a car in 5.3% of cases, as a motorcycle in 5.9%, and missed in 4.6%.

Detection ratios by class

Table 3.9 · Detected against true instances, per class

Class	True	Predicted	Difference	Implied factor

Class	True	Predicted	Difference	Implied factor
car	11,236	11,616	+3.4%	0.967×
heavy	**152**	**160**	**+5.3%**	**0.950×**
motorcycle	3,105	3,307	+6.5%	0.939×
All	14,493	15,083	+4.1%	**0.961×**

The heavy class has reversed. Under v1 it was under-detected by about 48% and needed a correction factor of 1.84× that varied by 57% between sessions. Under v2 it is over-detected by 5.3%, and all three classes sit within a 2.9% band. These are per-frame detection rates, though, not the whole-session counting factors in Section 4.4, which also depend on tracking, the displacement filter and counting only inside the region of interest (ROI). Section 4.5, not this table, decides whether heavy vehicles can be reported separately. What the table shows is that heavy vehicles are no longer misread as cars.

The remaining errors are of a type the counter absorbs

Table 3.10 · Remaining detection errors by type

Error type	Instances	Share
Cross-class misclassification	38	**0.26%**
Missed detections	865	5.97%
False positives on background	1,455	10.04%

A misclassification is permanent: a heavy vehicle counted as a car stays wrong. A missed detection or a false positive is temporary, and the counting pipeline in Section 4.1 is built to absorb both. It uses ByteTrack (Zhang et al., 2022), which links detections of the same vehicle across frames into a track with one ID.

- A vehicle missed in one frame keeps its track ID and is counted when it reappears.
- A false positive that does not form a stable track never reaches the count.
- Anything that does not travel 60 px is discarded regardless.

So v2 swaps errors the counter cannot fix for errors it mostly can. **That is why counting accuracy should improve by more than the 0.119 gain in mAP suggests.**

4 · Vehicle counting and validation

> **IN SHORT**
> The counter is within 1.7% of hand counts on total vehicles and 2.7% on cars + heavy, the group the simulation uses. Heavy vehicles are not reported separately, because the hand count and the model define "heavy" differently, not because the detector misses them.

Boxes on single frames are not yet a traffic count: that also takes tracking, counting only inside the ROI, and clear rules about what counts as a vehicle. This chapter sets out those rules and checks the result against hand counts.

4.1 Counter design

The counter (`detect_vehicles.py`), written by Ratchanon P. and Warachai A. as part of Module A, runs the detector on each frame, links detections of the same vehicle across frames, and counts each tracked vehicle once when it crosses the region of interest. It measures arrival rates offline from recorded video. The controller's live input is simpler: a count of vehicles at each signal head, once a second (Section 6.9).

Five design decisions set its accuracy. Most came from a failure we saw while checking against hand counts; the taxi filter is the exception, written before any count was taken.



Figure 4.1. The counter at work on approach A. Every box shows the detector's class, confidence score and track number. Green boxes are cars counted inside the lane region, grey boxes are outside it, and yellow boxes are motorcycles, which are tracked but not counted.

Tracking each vehicle under one ID

The first version removed repeat counts by distance, ignoring any detection within 30 pixels of one already counted. **It over-counted by about a factor of thirty**, because a vehicle moving across the frame kept leaving the 30-pixel circle and being counted again. The counter now uses ByteTrack (Zhang et al., 2022): a vehicle gets a track ID when first detected, keeps it across frames, and is counted once.

Counting windows in video time

Counting windows first moved forward by the computer's clock. At the roughly 4 frames per second the detector manages on the laptop used for offline counting (a T4 GPU runs it at about 70, Section 3.6), a 7-second window covered only about 0.43 seconds of video. Windows now move by **video time**, so a 7-second window covers 7 seconds of recorded traffic however fast the computer runs.

Class names read from the model

The counter first held a fixed list of three class names written for an earlier model. When the model was retrained with five classes, the list silently mislabelled some classes and dropped one. The counter now reads `model.names` when it loads the model, so it cannot disagree with the weights it is using.

Displacement filter for staged taxis

Taxis wait in three lanes on approach A (Section 2.5) without moving. The counter only counts a vehicle once it has moved **60 pixels** from where it was first seen, so waiting taxis are never counted, while a weaving motorcycle or a car that has driven into a queue is. We knew about the taxis from the interview and wrote this filter before any count was taken.

Confidence threshold

Each box comes with a confidence score from 0 to 1, and boxes below the threshold are dropped. Ours is **0.20**, taken from the peak of the F1 curve (0.205 for the first model) rather than a library default. F1 combines precision and recall into one score; plotting it against the threshold gives a curve whose peak is the best balance between the two. The training notebook redraws the curve for each new model so the value can be re-checked.

What the failures had in common

Appendix F lists five counter failures, and **none of them showed up in the model's own scores.** Each produced believable output, and we caught all five only by comparing against hand counts. One was a class-name table that quietly sent every heavy vehicle into the car count while the model scored mAP@50 of 0.766 on that class. That is the

main reason for the hand counts in Section 4.2: they are the only check in this project that can catch this kind of failure. The counter now prints its full class mapping at startup and warns about any name it cannot map.

4.2 Validation against hand counts

We ran the counter over the same footage and 15-minute window as a hand count and compared the two directly. The ROI spans the whole road, so both the straight-on and the turning movement cross it, and we compare the counter's total with the hand count for both together.

Table 4.1 · Detector against hand count, approach A, 15-minute window

Class	Manual	Detector	Difference	Accuracy
Total vehicles	298	303	+5	**101.7%**
cars + heavy	226	232	+6	102.7%
motorcycle	72	71	−1	98.6%
car	189	225	+36	119.0%
heavy	37	7	−30	**18.9%**
PCU-weighted flow	286.8	262.4	−24.4	91.5%

Vehicle counting accuracy is within 1.7% of the hand counts.

The `heavy` and PCU rows look alarming, but Section 4.5 explains them: the hand count and the model use different definitions. In the hand count, `heavy` means pickup and larger, while the v2 taxonomy maps `truck` to `car`. The PCU row is low for the same reason, and the hand count is the figure that is too high, because it weights those pickups at 2.0 instead of 1.0. Neither row is a detection failure; Section 3.7 shows heavy is detected at the same rate as every other class.

Why counting accuracy is better than mAP suggests

A mAP@50 of 0.883 might suggest counting errors of around 12%, yet the counter is accurate to 1.7%. mAP marks a box down if its IoU with the true box is 0.45 instead of 0.50, even when both mark the same vehicle in the same place. The counter only asks whether a tracked vehicle's centre crossed a polygon, so **a model that draws slightly loose boxes still counts correctly.** Counting accuracy is the measure this project depends on and the one a traffic engineer would ask for; we report mAP because it is the standard score for comparing with published work.

4.3 The one consistent error

Thirty heavy vehicles are missing from the detector count and thirty-six extra cars appear. The sizes match, which suggests heavy vehicles are filed as cars rather than missed, and the two errors almost cancel, which is why the total is within 1.7% while single classes are not. Section 4.5 shows across all five sessions that this is a **mismatch of definitions**, not the resolution limit of Section 3.2.

4.4 Calibration across sessions

To process the rest of the footage automatically, we worked out a correction factor per class: the hand count divided by the detector count for a session. Under the final model these cover **five sessions across three approaches and three times of day**, one session, one approach and one lighting condition more than the first model had.

Table 4.2 · Correction factors (hand count ÷ detector) across five sessions

Group	A_morn	A_mid	B_morn	B_mid	C_eve	Mean	Spread	Usable
cars + heavy	0.974×	0.969×	0.890×	0.887×	0.956×	**0.935×**	**9.4%**	**yes**
total	0.983×	0.948×	0.894×	0.853×	0.925×	0.921×	14.1%	marginal
motorcycle	1.014×	0.905×	0.908×	0.792×	0.867×	0.897×	24.7%	marginal
car	0.840×	0.742×	0.729×	0.621×	0.891×	0.765×	35.4%	no
heavy	5.29×	—	5.12×	13.50×	2.57×	**6.62×**	**165%**	**no**

Spread is (max − min) / mean. The `A_midday` heavy factor is undefined rather than large: the detector found no heavy vehicles where the hand count found 29, so that row's mean and spread use the other four sessions.

The group the analysis depends on improved: `cars + heavy` went from 0.917× at 12% spread under the first model to 0.935× at 9.4%, while the calibration set grew from four sessions on two approaches to five on three. A factor that tightens as new approaches and new light are added describes the detector, not one lucky sample. It is also the only group stable enough to apply widely, and the one the simulation needs, since the simulation has no motorcycles (Section 7.3). Heavy vehicles are still not reported separately: their spread rose from 57% to 165%, for the reason Section 4.5 gives.

The detector over-counts most in light traffic, from +1.7% at 298 vehicles to +17.2% at 128, which suits a capacity analysis built on peak conditions. Within each approach the lighter session always over-counts more, though not across approaches. A tracker losing queued vehicles and picking them up again is ruled out as the cause (Appendix C.1); Section 4.7 finds the real one.

The evening session extends calibration to dusk. Its `cars + heavy` factor is 0.956× against a daytime mean of 0.930×, a 2.8% difference (down from 6% under the first model), so automated counting of evening footage is validated too.

4.5 The heavy class: a mismatch of definitions, not a detection failure

Heavy vehicles cannot be reported separately at PCU 2.0, but not because the detector misses them. Under the final model the session-level heavy factor got worse while detection of single heavy vehicles got better, and one detection failure could not cause both. On annotated frames heavy is detected at 0.950×, within 2.9% of the other classes. Across five sessions the detector is short 128 heavy vehicles but has 181 extra cars, and its heavy share (2.0%) matches its training set (1.1%) while the hand count says 13.6% (Appendix C.2).

The cause is the taxonomy. Section 3.2 mapped `truck` to `car`, but the hand counts were recorded under the older five-class definition, where a pickup counted as heavy:

Table 4.3 · What "heavy" means in the hand count and in the model

	`heavy` means
Manual tally	pickup and larger
Model, v2 taxonomy	bus, lorry and larger

Vehicle counts are unaffected, because a pickup is one vehicle either way; that includes the arrival rates, the v/c table and the Bx = 1.51 finding. `cars + heavy` is unaffected too, which is why its spread is 9.4% against heavy's 165%. Only PCU flow from the hand counts is affected: it is too high by 5-25% per session and 13.7% overall (1,060.0 PCU counted against 932.0 corrected). Any hand-count `heavy` column in this report means pickup and larger. Re-counting one session under the three-class definition would settle it, and we recommend that before any PCU-weighted flow is published.

4.6 Platform drift

The drone holds position by GPS and by watching the ground, but it still drifts, moving the scene under an ROI drawn on one frame. Matching scenery landmarks against the first frame, one by one, gives 9-92 px of drift over 900 s, two to five times less than the original estimate (Appendix G).

We built a drift correction (`--stabilise`) and rejected it. It fits a similarity transform (shift, turn and scale of the whole image), but the real motion is a homography, which also captures the change of perspective as the camera tilts over a flat road, and the correction tripled the spread of the correction factors from 9.4% to 33.0%.

What matters is whether drift changes counts, so we tested that directly by counting `A_morning` twice, once with the ROI moved 50 px across the lanes, the most damaging direction and about the measured drift.

Table 4.4 · Count with the ROI as drawn and moved 50 px

	`cars + heavy`	**Factor**
ROI as drawn	232	0.974×
ROI displaced 50 px	**232**	0.974×

The 900 s count did not change; of 128 seven-second windows, 102 were identical and 26 differed by one vehicle. This is one session with a wide ROI (50 px is 6% of its width), so it shows drift of the measured size leaves counts unchanged on a wide approach ROI, not that counts can never be affected. Drift would matter only in the narrow far end of the ROI, which Section 4.7 shows is unreliable anyway. Every reported count uses the fixed ROI.

4.7 ROI depth: a known over-count we did not correct

Every correction factor in Section 4.4 is below 1.0, so the detector counts more than the hand count, and the cause is distance from the camera. On `B_morning`, 62% of detections come from the far third of the polygon, where the median vehicle is 58 px tall against 192 px near the camera. ByteTrack has least to work with there, so a track breaks and the tracker picks the vehicle up again under a new ID, counting it twice.

We recounted all five sessions with the ROI clipped where it narrows to half its widest. On the four free-flowing sessions this confirmed the cause: the mean factor was 0.996× at 4.1% spread, and `B_midday` detected exactly 86 vehicles against 86 counted by hand. `C_evening` went the other way, to 1.081×, because about half its vehicles queue above the cut: the full polygon counts vehicles arriving at the back of the queue, the clipped one counts them leaving past a line further down.



Figure 4.2. The clipping test on all five calibration sessions. Red is the ROI as drawn to the vanishing point. Green is the same ROI clipped where it narrows to half its widest. In C_evening, at the bottom, vehicles queue above the cut.

We did not adopt the clipping, and no figure in this report changes. A correction that fails on one session in five, for a reason that depends on where queues form, is not a correction. The working rule would be to clip the far end but never through a queue, which needs a judgement for each session. Appendix C.3 gives the per-session figures and two explanations we rejected.

5 · Field findings: demand, capacity and the central finding

> **IN SHORT**
> Bx needs 28.6 s of green but gets 20 s (v/c 1.51), while A, which conflicts with it, gets 40 s and needs 12.3 s (v/c 0.25). Moving 10 s of green from A to Bx brings Bx to v/c 0.95 and clears its queue, with no change to the cycle.

This chapter compares how much traffic arrives on each movement with how much green it gets. Every figure comes from the site or from footage recorded there, and where we had assumed a value first we show both, because in several cases the difference is itself a result.

5.1 Method

We counted vehicles two independent ways and used each to check the other.

Hand counts came from drone footage, using a tallying tool built for this project (`tally.html`), in 15-minute bins (the HCM's standard counting period), split by movement and lane, with motorcycles separate from cars and heavy vehicles. **A vehicle is counted when it crosses the stop line.** That rule matters here, because the taxis queuing across three lanes of approach A never cross the stop line and so drop out without any special case. Automated counts use the detector from Chapter 3 with the tracking described in Chapter 4, checked against the hand counts before use on the rest of the footage.

For capacity we report flow in passenger car units (car 1.0, motorcycle 0.33, heavy 2.0) alongside raw counts, because here they tell different stories. Movements B and A carry almost the same raw volumes (67 and 66 vehicles in comparable bins), but 31% of B's traffic falls in the hand count's heavy group against 20% on A. That group means pickup and larger (Section 4.5), so the real difference in load is smaller than these shares suggest.

5.1.1 Locally measured capacity parameters

We measured the saturation headway and startup lost time of Section 2.4 from `B_MORNING_DJI05` with a lap-timing tool built for this project (`headway.html`): five green phases that began with standing queues, 39 headway samples. The results were 2.31 s/veh against the HCM default of 2.00, and 2.95 s of startup lost time against 2.00.

The two pull in opposite directions. A 2.31 s headway, rather than the 2.50 s first assumed, makes discharge about 8% faster and raises capacity. But 2.95 s of lost time comes out of every green; on a 20-second phase that is 15% of the time. **On short phases the second effect is larger**, so the overall picture is worse than the original assumptions gave, as Section 5.4 shows.

5.2 Demand concentrates on the movement the signal does not control

The most consistent pattern in the data is the ratio between through traffic and traffic turning into the university.

Table 5.1 · Through traffic against traffic turning into ABAC, per session

Session	Day	Through	Turn into ABAC	Ratio
A_morning, 08:40	Monday	A = 66	**Ax = 232**	**3.52×**
A_midday, 11:37	Wednesday	A = 38	**Ax = 143**	**3.76×**
B_morning, 08:46	Tuesday	B = 133	Bx = 120	0.90×
B_midday, 11:50	Wednesday	B = 67	Bx = 61	0.91×

On the A approach **turning traffic outnumbers through traffic about 3.5 to 1**; on the B approach the two are about equal. Both ratios hold to within 7% while total volume doubles between midday and morning (on B, each morning volume is almost exactly twice the midday one). Four bins, two approaches and two times of day give two stable ratios, so the ratios belong to the junction, not to when we happened to sample.

The layout explains the imbalance

Both approaches lead to the same place, so the difference comes from the cost of each route. Ax is a left turn that crosses nothing and has no signal, so a driver making it never has to stop. Bx is a right turn across A's through traffic, so it is signalised. Given a cheap route and an expensive one to the same place, drivers coming from the east take the free left.

The busiest movement has no signal

That turning traffic is the largest flow is no surprise; that **the busiest movement has no signal** is the finding. An adaptive controller cannot give Ax more green because it has no signal to give it to. The only lever is indirect: clear the signalised movement that physically blocks it, faster. This limits what any signal-based change can do, and Section 5.7 shows the same pattern at the other end of the junction.

The guard's account matches. He waves left-turning vehicles in himself, he said, because straight-through vehicles stop and block the turn lane, and he estimated he does so 40-60% of the time. The measurements explain why that help is needed and why it falls to a person: the busiest movement has one lane and no signal, and it is blocked whenever the through movement queues. The interview and the data reached the same bottleneck independently.

5.3 The morning peak is later than the collection guide assumed

We ran the calibrated counter over all eight session videos (2,299 vehicles in about 2.5 hours) to get a demand profile across the day. Each approach had two flights, the second starting soon after the first ended.

Table 5.2 · Flow in the first and second flight: the peak ends after 09:00

Approach	First flight	Second flight	Change
A	830 veh/hr (08:40-09:05)	426 veh/hr (09:11-09:36)	**−49%**
B	791 veh/hr (08:46-09:06)	303 veh/hr (09:13-09:28)	**−62%**

Both approaches roughly halve within minutes of 09:00, and counts rise within the first flight (131, then 171, then 175 vehicles per 450 seconds): demand climbs towards 09:00 and then collapses. So the morning peak here is about 08:15-09:00, not the 07:00-08:30

the data collection guide assumed. It follows class start times rather than general commuting, which fits a junction serving mostly the university on a road that dead-ends 9 km further on, and it means the first-flight measurements do capture real peak conditions.

It also explains something we saw separately: the university closes the Lane C exit for 15-30 minutes between 08:30 and 09:00, **exactly on the measured arrival peak.** The closure is aimed at the moment inbound demand surges.

5.4 The central result: Bx is starved while A is over-served

Here we compare the green given with the green needed for the two conflicting movements on approaches A and B. Every input is measured: arrival rates from Section 5.2, signal timing from Chapter 2 and saturation flow from Section 2.4. Arrivals per cycle (queueing arrivals) are the vehicles that arrive during one cycle and join the queue.

Movement Bx, the signalised right turn into the university

Table 5.3 · Bx per cycle: arrivals exceed what the green discharges

Queueing arrivals per cycle	**11.1 vehicles**
Green time given	**20.0 s**
Usable green after 2.95 s startup lost time	17.1 s
Discharge achieved	**7.4 vehicles**
Green time **needed**	**28.6 s**
Deficit	**8.6 s**
Volume-to-capacity ratio	**1.51**

With v/c above 1.0 the queue never settles; here it grows by **3.7 vehicles every cycle.** The B approach holds about 12 vehicles before the queue reaches the four-way junction 78 m upstream, so at 3.7 vehicles per 107-second cycle the storage is gone in 5.7 minutes and the queue starts blocking a second junction. That matches the field observation that the Bx queue "almost" reaches the four-way: it overruns on a clock of about six minutes and is partly relieved by vehicles using the emergency lane and by guards waving traffic through.

Movement A, the through movement it conflicts with

Table 5.4 · A per cycle: green far exceeds need

Arrivals per cycle	8.1 vehicles across 2 lanes
Green time **needed** (including lost time)	**12.3 s**

Green time given	**40 s**
Surplus	**27.7 s**
Volume-to-capacity ratio	**0.25**

A gets more than three times the green it needs while the movement it conflicts with spills back into a neighbouring junction.

The remedy in numbers

Because A and Bx conflict, green can move between them without changing the cycle or the phase structure.

Table 5.5 · Bx v/c for each amount of green moved from A

Green shifted A → Bx	Resulting Bx v/c	Outcome
5 s	1.17	queue still grows
8 s	1.03	queue still grows
10 s	**0.95**	**queue clears**
12 s	0.88	clears comfortably

Moving ten seconds of green from A to Bx removes the oversaturation. A keeps 30 seconds against a need of 12.3, staying well below capacity at a v/c of about 0.34.

The effect of measuring the parameters locally

Before we measured startup lost time, the HCM default of 2.00 s put the needed transfer at 8 seconds.

Table 5.6 · How each measured parameter changed the Bx diagnosis

Method	Discharge per cycle	Bx v/c
Original assumptions (2.50 s/veh, no lost time)	8.0	1.39
Measured headway only	8.7	1.28
Measured headway and measured lost time	**7.4**	**1.51**

Measuring locally made the diagnosis **worse** and moved the remedy from 8 seconds to 10. A measurement that makes your own result worse is a reason to trust it.

5.5 The exit movements: Lane C

We measured the two exit movements from footage taken at the gate on Monday 24 August, 16:16-16:41, over one 15-minute bin. From inside the gate the turn direction is visible, so the split is **measured, not assumed**.

Table 5.7 · Exit counts at Lane C, by movement and class

Movement	Car	Moto	Heavy	Total	Interval (cars + heavy)

Movement	Car	Moto	Heavy	Total	Interval (cars + heavy)
CB: exits ABAC turning west, signalised	98	52	9	**159**	**8.4 s**
CA: exits ABAC turning east, free flow	58	33	9	**100**	**13.4 s**

The split is 61:39 in favour of CB, and motorcycles are 33% of both.

CB sits exactly at capacity

Table 5.8 · CB per cycle: exactly at capacity

Queueing arrivals per cycle	12.6 vehicles
Green time given	31 s
Usable green after startup lost time	28.1 s
Discharge achieved	12.1 vehicles
Green time **needed**	**32.1 s**
Volume-to-capacity ratio	**1.04**

CB queues on most cycles and clears just in time. Unlike Bx its queue does not grow without limit, but it has no margin: any rise in demand or disruption to discharge tips it into oversaturation.

First check of the detector on evening footage

This session was also the first test of the counter's correction factors in evening light. When we first made the check, with the first model, the detector had never seen evening footage; the figures below are from the final model, whose training set includes evening sessions (Section 3.3).

Table 5.9 · Detector against hand count on evening footage

Group	Manual	Detector	Evening factor	Daytime mean
cars + heavy	174	182	**0.956×**	0.930×
total	259	280	0.925×	0.920×
motorcycle	85	98	0.867×	0.905×
heavy	18	7	2.57×	7.97×

The `cars + heavy` factor holds, 0.956 against 0.930, a **2.8%** difference, which is why we could count the rest of the evening footage automatically. The motorcycle factor moves only from 0.905 to 0.867 (4.1%), within the daytime spread; under the first model it fell from 0.91 to 0.76, which we put down to small motorcycles in low light, and the final model largely removes that. So the case for reporting `cars + heavy` now rests on Sections 4.4 and 4.5, not on evening light. The heavy factor (2.57× against 7.97× by day) is the definition mismatch of Section 4.5: the evening session simply has fewer pickups.

5.6 The shared lane: how blocking occurs

Approach A has two lanes. The inner lane, beside the median, is for straight-through traffic only; the outer lane is shared between straight-through traffic and the left turn into the university. A turning driver has no choice of lane, while a straight-through driver can use either. Ax has no signal, so turning drivers never need to stop, but one straight-through car waiting at red in the shared lane blocks every turning vehicle behind it, holding them at a signal that does not apply to them.

We measured how often straight-through drivers pick the shared lane, from `A_morning` footage, counting only straight-through cars (turning vehicles have no choice, and motorcycles filter around stopped traffic so neither block nor get blocked).

Table 5.10 · Lane chosen by straight-through cars on approach A

	Count	Share
Straight-through cars observed	56	
Inner lane	53	94.6%
Shared lane	**3**	**5.4%**

Table 5.11 · Shared-lane use: assumed, inferred and measured

Source	Shared-lane share
Simulation's original working assumption	50%
Inferred from the "blocked 40-60%" field impression	13-23%
Measured	**5.4%**

Drivers avoid the shared lane far more than either estimate supposed: about nineteen in twenty stay beside the median.

Late merging

All three shared-lane cars stayed in the inner lane until about five car lengths before the turn, then merged late. A late merger enters at the head of the queue, so one car stops every turning vehicle behind it for the rest of the red. These events are rare, but each has close to the largest possible effect.

Reconciling 5.4% with a 40-60% impression

The measured 5.4% only seems to contradict the reported blocking frequency. Modelling every car arriving across the red as an independent random (Poisson) arrival answers the wrong question. **One straight-through car in the shared lane is enough** to cause blocking, and such cars merge at the front, so the question is whether any straight-through car picks the shared lane in a given cycle. At about 5.4 straight-through cars per cycle:

```
P(at least one in the shared lane) = 1 − (1 − 0.054)^5.4  =  26% of cycles
observed directly                  = 3 blockers in 10.3 cycles  =  29%
impression from daily driving      = 40–60%
```

A computed 26% and an observed 29% are the same order as a 40-60% impression, and an impression formed while driving gives extra weight to memorable events. The mechanism fits all three.

What this means for modelling

A model that spreads straight-through traffic evenly across the shared lane produces frequent small delays. The real junction has **infrequent, severe blocking events.** They give the same average delay but very different queues, and it is the severe events, not the average, that push the Bx queue back towards the four-way junction. Chapter 6 depends on this.

5.7 The same failure appears at both ends of the junction

The same arrangement, a free-flow movement sharing a lane with a signalised one, occurs at the exit. We saw CA blocked seven times in the 15-minute Lane C bin, which at a 107-second cycle is 8.4 cycles.

Table 5.12 · The same blocking at the entry and at the exit

	Free-flow movement	Blocked by	Frequency
Entering	`Ax`, the left turn in	`A` queueing in the shared lane	**26-29%** of cycles
Exiting	`CA`, the left turn out	`CB` queueing in the shared lane	**83%** of cycles

In both cases a movement that never needs to stop is held at a signal meant for another. The exit is far worse because CB runs at a v/c of 1.04 and queues on nearly every cycle, while only 5.4% of A's through traffic uses the shared lane at all. We also saw drivers bound for CA wait and then turn two at a time once CB gets green, using the width of the exit to get round the queue: an informal habit formed to work around the layout.

What this means for the recommendation

Because it happens wherever a free-flow movement shares a lane with a signalised one, **this is the junction's typical way of failing**, not a quirk of one approach. It also tightens the limit from Section 5.2: an adaptive controller can only help Ax and CA by clearing what blocks them. So extra green for CB is not only for CB's own demand. It is the only lever for unblocking CA, which carries another 100 vehicles per 15 minutes with no signal of its own.

5.8 Every signalised movement, measured

Table 5.13 · Green given against green needed, every signalised movement

Movement	Green given	Green needed	v/c	Status
A	40 s	12.3 s	**0.25**	under-used: over three times the green it needs
B	65 s	—	**0.25**	under-used
Bx	20 s	28.6 s	**1.51**	**oversaturated**: queue spills toward the four-way
CB	31 s	32.1 s	**1.04**	**at capacity**: queues on most cycles

> **Both movements serving the university are at or beyond capacity. Both through movements hold more than three times the green time they require.**
>
> Every figure in this table was measured on site.

That is the main argument of this project, and Section 5.4 shows that moving ten seconds of green between two conflicting movements fixes the worse of the two.

We also checked the figures at movement level on approach B, where B and Bx use separate lanes and can each have their own ROI. A split run over the same 900 s reproduces the hand-counted 52:48 division exactly, with a combined correction factor identical to the single-ROI factor (Section 6.9). So where movements are physically separate, the detector is checked at movement level. A and Ax share one lane (Section 5.6) and cannot be separated, so their split rests on the hand count alone.

5.9 Visual confirmation: the result in a single frame

Footage `DATA_LANE_B_MORNING/DJI_20260714084627_0005_D.MP4` at 01:24 shows the whole finding without any analysis: Bx queued seven deep in the median-side lane, waiting to turn into the university, and B's outer through lane on the same approach, under the same green, completely empty. Two movements share one road and one stage; one is saturated and the other unused, as v/c values of 1.51 and 0.25 predict. The same thing showed while counting: **queues rarely form in both lanes at once.**



Figure 5.1. B_morning at 01:24. Bx is queued seven deep in the median-side lane. The outer through lane for B, on the same approach and under the same green, is empty.

One frame could catch an unlucky moment, so we checked the frames either side.



Figure 5.2. The same approach at 01:18 (left) and 01:30 (right). The imbalance holds across the phase, not just for an instant.

Across twelve seconds the median-side lane stays loaded and the outer lane stays empty. **The imbalance is how this approach normally runs, not a trick of timing.**

The fault can be corrected within the existing plan by moving ten seconds of green from A to Bx. An adaptive controller should find that without being told; Chapter 6 tests whether it does.

6 · Modules B and C: adaptive control and simulation

IN SHORT
SODA reads the vehicle count at each signal head once a second, serves the queue that formed first, and sizes each green from the count. Over 108 paired simulation runs, mean wait per vehicle went from 44.15 s to 38.44 s (−12.9%), worst-case wait changed by −41.6% and wait on Bx by −53.9%. In the evening peak its average wait was 6.8% higher than the fixed-time plan's (p = 0.024).

Sections 6.1 to 6.5 describe the control algorithm, SODA (Sampled-Observation Demand-Adaptive control), written by Ratchanon P., Warachai A. and Badin B. (Module B). Sections 6.6 to 6.8 describe the simulation, its test scenarios and the statistical analysis, built by Badin B. (Module C). Section 6.9, the interface between the modules, is shared work, Section 6.10 reports how we checked the simulation against the site, and Sections 6.11 to 6.17 give the results. Once a second, SODA reads how many vehicles wait at each signal head, serves the queue that formed first, and sizes the green from that count.

6.1 Controller architecture

SODA does not rotate through phases on a timer. Once a second it takes a reading: one count per signal head, the number of vehicles inside that head's region of interest. It reads nothing else, no vehicle identities, arrival times or speeds, and every decision comes from those counts.

The reading interval was 2 seconds at first, set by what the detector pipeline could keep up with rather than by traffic. We changed it to 1 second on 14 September 2026 after a paired test (Appendix D.4): reading every second shortened the mean wait by 0.42 s in MorningPeak and 1.60 s in MiddayNormal, while reading every 3 s made no difference in MorningPeak and added 0.73 s in MiddayNormal. The gain comes from gap-out, ending a green once every lane it serves reads empty twice in a row (Section 6.3), firing sooner. Reading faster than once a second is ruled out because real detection misses come in runs: a car hidden behind a truck stays hidden for a second or more, so two readings half a second apart would often both miss it and gap-out would end the green on a lane that is not empty. All the results in Sections 6.11 to 6.17 were produced at 1 s.

Demand starts a round, and each round passes through three states: ALL_RED_IDLE (no demand, every signal red), COLLECT (wait up to 7 s for demand to build) and SERVE (give greens).

```
ALL_RED_IDLE → COLLECT → SERVE → back to ALL_RED_IDLE when no demand
remains
```

6.2 Two separate rules: which head goes, and for how long

SODA keeps two questions apart.

Order comes from how long a queue has existed. Each head has a queue clock, `queueStart`, set at the reading on which the head goes from empty to occupied and cleared whenever it reads empty. The next green goes to the head whose queue formed earliest, however many vehicles it holds; an exact tie goes to whichever head comes first in the junction's existing phase order, A, B, Bx, CB.

Duration comes from the length of that queue, using the capacity rule below.

Keeping the rules apart matters. Ordering by demand alone would starve a lightly loaded approach, leaving it waiting without limit, while giving every movement equal time would repeat the fixed plan's waste.

The proposal's formula, tested against measured demand

The project proposal sized each movement's green by its share of total demand:

```
G(i) = G_min + (G_max − G_min) × D(i) / ΣD
```

with `G_min = 8 s` and `G_max = 45 s`, both chosen before any measurement existed. Tested against the arrivals per cycle from Sections 5.4 and 5.5 (A 8.1 vehicles across two lanes, Bx 11.1 across one, CB 12.6 across one), it fails:

Table 6.1 · The proposal's share formula against the green each movement needs

Movement	Demand share	Formula allocates	Green required	Shortfall
A	0.255	17.4 s	12.3 s	over-served by 5.1 s
Bx	0.349	**20.9 s**	**28.6 s**	**short by 7.7 s**
CB	0.396	22.7 s	32.1 s	short by 9.4 s

The fixed plan gives Bx 20.0 s and the share formula would give it 20.9 s, repeating almost exactly the failure this project sets out to fix. The formula knows nothing about how many lanes serve a movement or how fast a standing queue leaves: A carries 4.05 vehicles per lane and Bx 11.1 in one lane, yet the formula gives A 17.4 s and Bx 20.9 s. Changing the bounds does not help; lowering `G_min` makes Bx worse, and `G_max` would have to reach 70 s, an unbelievably long green for a single-lane turn, before Bx got enough (Appendix D).

The implemented rule: capacity-based allocation

The fault is in the form of the equation, not its settings, so `AdaptiveController.cs` shares green by capacity instead, using the two values measured in Section 2.4:

$$\mathbf{G_raw = \ell_1 + D \times h}\text{, then }\mathbf{G = clamp(\ G_raw\ ,\ g_min\ ,\ g_max\ )}$$

where $\boldsymbol{\ell_1 = 2.95}$ **s** is the measured startup lost time and **h = 2.31 s** the measured saturation headway. (6.1)

Clamping keeps the result between g_min = 8 s and g_max = 45 s. Each green covers its own startup lost time plus the time its queue takes to discharge at the measured rate, and the rule reproduces all three Chapter 5 figures exactly (A 12.3 s, Bx 28.6 s, CB 32.1 s), as it should, since Chapter 5 uses the same values.

When several heads are green together, the fullest lane sets the green. Heads served at once (A, A2 and B in stage 1) discharge side by side, so the phase lasts until the longest queue has cleared:

$$\mathbf{G = clamp(\ max\ over\ lanes\ in\ group\ (\ \ell_1 + D_i \times h\ )\ ,\ g_min\ ,\ g_max\ )} \quad (6.2)$$

Taking the largest lane rather than averaging matters because queues here are rarely even: only 5.4% of straight-on traffic on approach A uses the shared kerb lane (Section 5.6). SODA reads the count `D` when the green actually opens, after any amber and all-red, because at peak a count taken 5 s earlier is already two or three vehicles out of date. The C# code is in Appendix D.

Read backwards, the same equation gives how many vehicles a given green can discharge, which is how the evaluation compares the two controllers:

$$\mathbf{N = (\ G - \ell_1\ )\ /\ h = (\ G - 2.95\ )\ /\ 2.31} \quad (6.3)$$

Table 6.2 · Vehicles each fixed phase can discharge at the measured headway

Fixed phase	Green	Vehicles discharged
P1 · A + A2 + B	40 s	16.0
P2 · B + Bx	**20 s**	**7.4**
P3 · CB	31 s	12.1

Bx discharges 7.4 vehicles per cycle against 11.1 arriving: the v/c of 1.51 in Section 5.4, reached here from the signal plan instead of from demand.

The role of g_min and g_max

In this rule the bounds are safety limits, not the sharing mechanism. `g_min` guarantees a green safe for drivers and pedestrians, and `g_max` stops a saturated movement taking over the cycle when its queue never ends. They only bind at the extremes, since the formula is a straight line from D = 3 to 18 vehicles, the whole demand range measured here (Appendix D). At 8 s, startup lost time takes 37% of the minimum green before any vehicle crosses the line, and the capacity rule shows that loss as its own term instead of hiding it.

None of this was visible before the field measurements. A controller built on the share formula and tested on invented arrival rates would have seemed to work, because green would visibly follow demand. The capacity analysis in Chapter 5 and the controller code reached the same correction separately, which is the best evidence we have that it is right.

6.3 Ending a green early: the calculated green is a ceiling

The calculated G is an upper limit, not the time actually served. Three conditions can end a green sooner:

$$\mathbf{G_effective = min(G , t_gap\text{-}out , t_starvation , t_max\text{-}green)} \tag{6.4}$$

SODA has two anti-starvation rules so that no lane is ignored for long. The interrupt cuts a green; the override moves a long-waiting head to the front of the next decision (Section 6.4). Both are fractions of `maxWaitTime`, a 45 s waiting limit: the interrupt fires at 0.90 × 45 s = **40.5 s** (only once the current green has run 20 s), the override at 0.95 × 45 s = **42.75 s**. Where those two numbers appear later, they are these two rules. Max continuous green stops any group holding green longer than 90 s while others wait.

Table 6.3 · The three rules that end a green early

Mechanism	Ends the green when	Parameter
Gap-out	every head in the group reads zero on 2 consecutive readings	fires 2 s after the last non-zero reading, at the 1 s reading interval used in the reported batch
Anti-starvation interrupt	green has run ≥ 20 s **and** a head outside the group has waited ≥ 0.90 × max wait	20 s / 40.5 s
Max continuous green	the same group has held green continuously while traffic waits outside it	90 s

Gap-out counts readings, not seconds, so one missed detection cannot end a phase. The interrupt cannot cut a green before 20 s, which clears (20 − 2.95) / 2.31 = **7.4 vehicles**, so no phase is cut too short to be useful.

6.4 Decision sequence

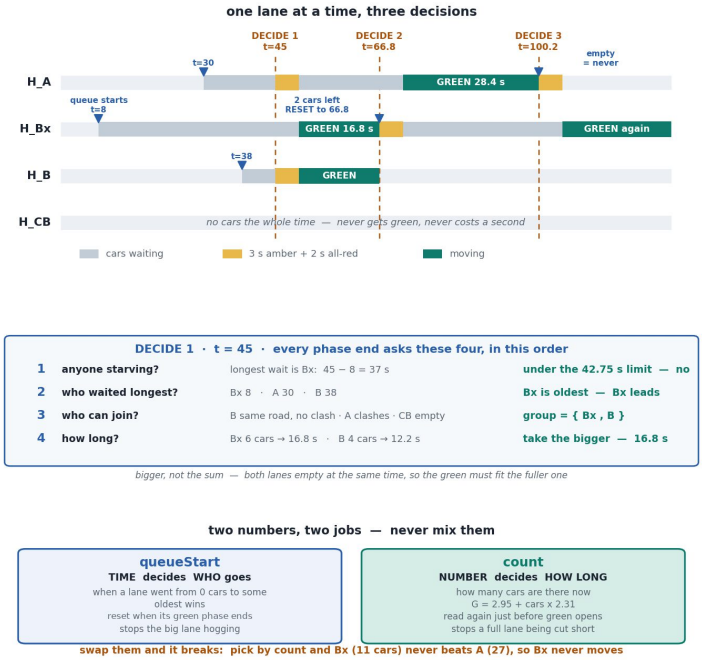


Figure 6.1. One worked example of the controller. Top: three decisions across four signal heads, with amber and all-red paid only by heads that stop. Middle: the four questions asked at the first decision. Bottom: the queue clock decides which head goes, and the count decides for how long.

The green-time formula is one step of each round. Figure 6.1 shows the point that is easiest to lose: the count `D` is the only value in the equation and decides *how long*, while the queue clocks and waiting times never enter the formula and decide *which* heads go and in what order. A description of SODA that gives only the equation has described the timing and left out the scheduling.

1 · All-red idle. With no demand on any head, every signal is red and SODA waits.

2 · Collect. When a head becomes occupied, SODA waits up to 7 s for demand to build. The 7 s is a limit, not a fixed wait: the window ends at once when two or more heads are occupied or any head holds three or more vehicles. So more traffic means an earlier decision, and only a lone vehicle at an empty junction waits the full 7 s. The limit fires at exactly 7.0 s, not at the next reading.

3 · Select a leader. Starvation is checked first: any head that has waited $\geq 0.95 \times$ max wait (42.75 s) leads at once, longest wait first, and SODA logs the event. Otherwise the head whose queue clock started earliest leads (Rule 1), not the one with the most vehicles, since choosing by volume would leave light movements such as the shared-lane traffic waiting without limit. An exact tie goes to the head first in the order A, B, Bx, CB.

4 · Build the green group. Starting from the leader, SODA adds other occupied heads in queue-clock order as long as each conflicts with nothing already in the group. Heads sharing a physical signal (A and A2) always go together whatever their counts; otherwise SODA could not see vehicles arriving into a lane that is already green.

5 · Clearance. Amber 3 s then all-red 2 s, applied only to heads that must turn red. A head that is green and stays in the new group keeps its green without a break. The fixed plan pays the full 5-second intergreen at every stage change; SODA pays it only for heads that change state and saves 5 s whenever consecutive groups overlap.

6 · Serve green, with G computed as in Section 6.2 from a fresh count once clearance is over, while the early-ending checks of Section 6.3 run throughout.

7 · End the phase. Every served head has its queue clock reset, to the phase-end time if it still holds vehicles or cleared if it is empty, and SODA returns to step 3, or to idle if no head has demand.

Step 7 is what makes a saturated junction take turns instead of serving one head for ever. Without it, a head that never empties (Bx runs at v/c 1.51) would keep its first queue clock and win every decision. The simulation first applied the reset only to heads turning red, so a head that stayed green across phases kept its clock; Section 7.4 describes that fault and its effect.

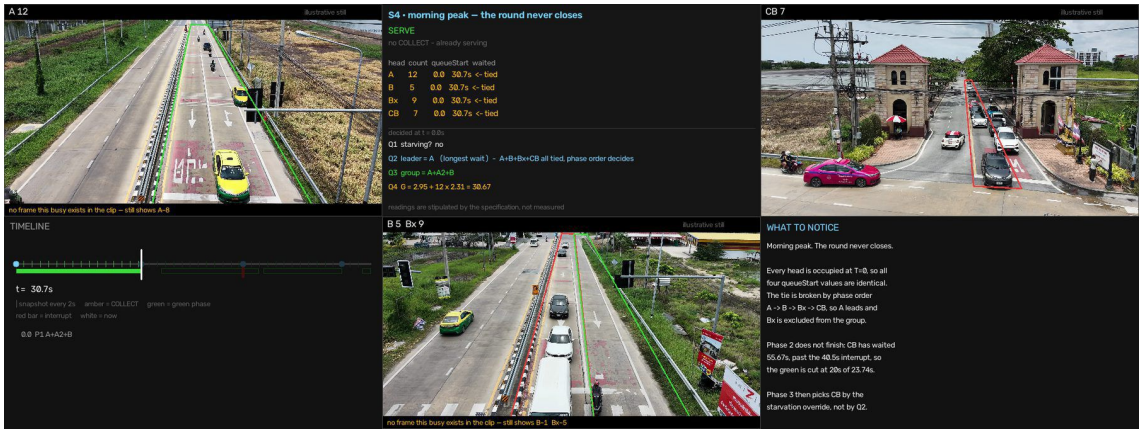


Figure 6.2. One decision round from the reference trace (scenario S4, morning peak). The camera panels show each head's count, the centre panel shows the four questions answered in order, and the notes panel explains the choice. The values are illustrative, set by the specification rather than measured.

Starting from cold

`AdaptiveController.Start()` sets every signal to red, the idle state of step 1, and the first head to register a vehicle starts the first round; the starvation override cannot fire because nothing has waited yet. We start from all-red on purpose. Guessing a starting green, the last one used or a fixed default, would give right of way to an approach

SODA has not yet seen. With all-red, the only cost falls on the first vehicle of the day, which SODA sees at the next reading and which, if alone, waits out the COLLECT window. That vehicle is not queued behind anyone.

6.5 Conflict matrix and which movements can go together

SODA only serves movements together if they do not conflict.

Table 6.4 · Movement groups SODA can serve together

Group	Movements served together
1	A + B
2	B + Bx
3	CB alone

Table 6.5 · Conflicting movement pairs, and why they conflict

Conflicting pair	Why
A and Bx	Bx turns right across A's path into the university
A and CB	CB crosses A's lanes
B and CB	CB merges into the lane B uses
Bx and CB	their paths cross in the middle of the junction
A2 and Bx	A2 shares A's signal head, so it conflicts with whatever A conflicts with
A2 and CB	A2 shares A's signal head, so it conflicts with whatever A conflicts with

A and A2 are always green together. We designed this matrix before fieldwork, and watching the existing signal later **confirmed it three separate times** (Section 2.2): A ran with B, Bx joined B part-way through B's green, and CB ran alone.

6.6 Simulation environment

Badin B. built the simulation in Unity 6 (6000.5.4f1) on the Built-In Render Pipeline, with its behaviour written as C# `MonoBehaviour` components.

It is a behavioural digital twin: built to behave like this junction in how queues form and leave, not to look like it. Vehicles are simple rule-following agents, each carrying car-following, queue discipline, a stop line and a signal state. They are not Unity `NavMeshAgent`s, because path-finding is not the problem being modelled and navigation-mesh agents add collision avoidance that a queue at a stop line does not show. The layout is built to scale from the measurements in Chapter 2 by one Editor script

(`ABACSceneBuilder`) that generates every object from named constants, so a correction from field data means changing one number and rebuilding, not editing a scene file by hand.

Each vehicle gets a driver temperament: aggressive 25%, normal 60% and cautious 15% (computed as the remainder, so the three always add to 100). Each has its own following distance, reaction time, braking and readiness to run an amber, which spreads waiting times within a lane. Cars keep a following distance that grows with speed, and at green each driver waits to see the car ahead move first, so a queue pulls away car by car as it does on the road (Section 6.11.3 explains why this matters). An A2 car joining the shared kerb lane waits for a gap of at least 1 s before it merges, and a car that stops short of the line at red creeps up to it; Section 6.11.2 describes the defect these rules replaced.

The two controllers run on identical traffic. Each lane has its own seeded `System.Random`, never Unity's global generator, and every random draw happens when a vehicle arrives, never depending on the traffic state. So the fixed-time and SODA runs of a scenario see the same vehicles at the same moments, and any difference comes from the signal logic alone. Each pair's seed is a stable FNV-1a hash of the scenario name plus 31 × the repetition number, so seeds hold across sessions and machines.

Seven lanes are modelled, including A2, which is driven by A's signal head and shares one ROI object with Ax, as a single real detector on that lane would. Ax has no phase of its own; it is held back through `lightIsInverted`, which makes it read Bx's signal in reverse (Bx green or amber means Ax red, since both turn into the university from opposite directions). Both mechanisms are in the simulation as of commit `7911b30` (4 September) and carry out the shared-lane behaviour of Section 5.6.

The fixed-time plan in the simulation runs the stages measured on site (Section 2.2):

Table 6.6 · Fixed-time stages as run in the simulation

Stage	Green movements	Green
1	A, A2 and B	40 s
2	B and Bx	20 s
3	CB	31 s

Each stage ends with 3 s of amber and 2 s of all-red, 15 s over the three stages, for a 106 s cycle, within about a second of the 107 s measured on site.

Automated tests

Eleven automated test suites check the code outside Unity: that conflicting heads are never green together, that no lane can be locked out for ever, that the car model discharges a queue at the measured rate, and that the scene's settings match the code's,

among others. They guard against real bugs found during development, including the queue-clock fault in Section 7.4 and a Unity setting that silently kept old values in the scene file. SODA also checks its two measured constants at start-up: if the scene carries values other than 2.95 s and 2.31 s, it restores them and logs an error.

6.7 Test scenarios

Scenarios come in two kinds, and the kind decides how a result may be read. Scripted scenarios (T1-T7) release a fixed list of vehicles at fixed times. They repeat exactly, so each one demonstrates a mechanism rather than producing statistics:

Table 6.7 · Deterministic test scenarios and what each demonstrates

	Scenario	Demonstrates
T1	single lane, 10 vehicles	baseline discharge
T2	all lanes equal	symmetric allocation
T3	`Bx` against `CB`	the conflicting-movement case
T4	starvation test	the anti-starvation interrupt
T5	ABAC entry and exit	the site's dominant pattern
T6	free movements only	`Ax` and `CA` in isolation
T7	driver-behaviour comparison	sensitivity to the driver mix

Poisson scenarios (T8-T12) draw random arrivals at a set mean interval per movement and are the ones used for statistics. During development they used five synthetic demand levels:

Table 6.8 · Synthetic demand levels used during development

Level	Mean interval
very high	1.5 s
high	2.5 s
medium	4 s
low	7 s
very low	15 s

T8 MorningPeak, T9 EveningPeak, T10 MiddayNormal, T11 ABACEvent and T12 Night each gave one level to each of the six movements, and a runtime scenario builder sets counts, spacing, start times, mean interval (with a vehicles-per-hour readout), driver mix and duration. Measured rates replaced the synthetic levels wherever a measurement exists (Section 6.9); elsewhere the rate is an estimate (Section 7.3).

The scenarios in the final batch

The final batch used four Poisson scenarios; T12 Night was not part of it.

Table 6.9 · The four scenarios in the final batch and where their rates come from

Scenario	What it represents	Where its rates come from
MorningPeak	heavy traffic into the university	measured from drone footage, except the exit movements (Section 6.9)
MiddayNormal	ordinary traffic on every approach	measured from drone footage (Section 6.9)
EveningPeak	heavy traffic out of the university	partly measured (CB and CA, Section 6.9), partly estimated
ABACEvent	heavy traffic in and out at once, as on graduation day	estimated; scaled down on 18 September 2026, before the final batches, after the first version carried more traffic than the junction could physically handle

MiddayNormal is a what-if scenario: the real signal flashes amber from 09:00 to 15:00 (Section 2.3), so at midday neither controller actually runs on site.

A run is one simulation of one scenario under one controller, a pair is the fixed-time run and the SODA run sharing a seed, and a batch is the full set. Each scenario ran 27 times under each controller: 4 scenarios × 27 repetitions × 2 controllers = 216 runs, or 108 pairs. Each run simulates 1,800 s of traffic after a 120 s warm-up that is simulated but not measured, with SODA reading every 1.0 s. Unity runs at ten times real speed; runs at 1×, 5× and 10× agree within normal run-to-run variation. Section 6.11.1 gives the full make-up of the batch.

6.8 Statistical analysis

Within a scenario and seed both controllers see byte-identical arrivals (Section 6.6), so the study uses a paired design: each fixed-time run has one SODA partner that met the same traffic, and what we test is the difference within each pair. That is why the random arrivals are kept independent of the traffic state.

Each run writes three log files, one row per vehicle, one per run and one per signal phase; Appendix D.5 lists their columns. Each vehicle's movement is recorded, so Ax and A2 stay separate even though they share one ROI. Waiting times come from the simulator's own record of when each vehicle leaves its region, which SODA never reads: SODA is limited to counts, but the measurement is not.

Waiting times are right-skewed (most are small, a few are very large) and cannot fall below zero, so they may not follow a normal distribution. Every per-run comparison therefore reports two tests: the paired *t*-test, which assumes the differences are normally distributed, and the Wilcoxon signed-rank test, which uses ranks and does not. A

Shapiro-Wilk test of the differences says which to rely on. Each comparison also gives the mean difference, its 95% confidence interval and Cohen's *d*. Every difference in this chapter is SODA minus the fixed-time plan, so a negative value means SODA was faster.

The headline figure, the mean wait per vehicle, pools vehicles across runs, so it is tested with a paired bootstrap: whole pairs are resampled thousands of times and the per-vehicle difference recomputed each time, giving its confidence interval and *p*. The *t*-test and Wilcoxon test are used for quantities measured once per run, such as the worst-case wait and the number of vehicles cleared. Mean wait and worst-case wait are both main results (Sections 6.12 and 6.13), because this junction's failure is a problem of the tail: the few vehicles that wait far longer than average.

A deadlock is a run in which the junction locks up and vehicles stop leaving. The simulation has no deadlock detection or recovery; whether a run jammed is judged afterwards from its log files. Every figure in Sections 6.11 to 6.17 is computed directly from the batch files by a script (`fill_results.py`), not typed by hand.

6.9 Interface between the modules: measured arrival rates

Module A feeds the rest of the system in two ways. Offline, the counter (Chapter 4) measured how often vehicles arrive on each movement, and those rates drive the simulation. Live, a fixed camera would give SODA one count of cars and heavy vehicles per signal head every second, which needs no tracker, only a count of what is inside each region at that moment. In the simulation SODA reads exact counts from the simulated regions instead (Section 7.3).

Rates are given as a mean interval in seconds between vehicles. Feeding a rate where an interval is expected turns demand upside down, and the output still looks believable.

```
// MEASURED — manual tally, cars + heavy, 15-minute HCM bin
// order: (A, Ax, B, Bx, CB, CA)
Poisson("T8 · MorningPeak",   "measured", 18.8f, 5.1f,  9.0f,  9.6f,  —,    — )
Poisson("T9 · EveningPeak",   "measured",   —,     —,     —,     —,  8.4f, 13.4f)
Poisson("T10 · MiddayNormal", "measured", 29.0f, 9.7f, 20.5f, 21.4f,  —,    — )
```

These are hand-count figures, not detector output. Each can be rebuilt from the tally files in one step, and Chapter 5 uses the same figures; feeding the simulation a different set would leave Chapters 5 and 6 describing two different junctions. The detector could not give the A/Ax split anyway, because A and Ax share one lane (Section 5.6). B and Bx can be separated: a split run over the same 900 s gives the hand-counted 52:48

division exactly, at a combined correction factor of 0.890x, the same as the single-region factor. **The central result does not depend on this choice**: a detector-derived set agrees with the hand-count set on Bx within 0.3% (9.57 against 9.60). Appendix A gives the detector-derived set and the full reasoning.

`CB = 8.4` and `CA = 13.4` are evening figures, because the drone recorded one approach at a time and the morning session has no exit measurement. We later measured midday CB and CA by running the detector with one region per movement on the midday Lane C footage (CB 10.1 s, CA 13.2 s); the morning figures cannot be recovered, so a morning scenario cannot use measured exit rates. A and Ax were recorded on a Monday, which the guard describes as a low-volume day, and we apply no correction because his "×0.7" factor was never measured; 18.8 and 5.1 are therefore below the true peak. B and Bx were recorded on a Tuesday, at full peak. Motorcycles are left out because the simulation models cars only (Section 7.3).

Three further measured values shape the simulation:

Table 6.10 · Further measured values built into the simulation

Parameter	Measured value	Section
Shared-lane usage by through traffic	**5.4%**, merging late at the queue head	5.6
B approach storage before spillback	**12 vehicles / 78 m**	2.1
Saturation headway and startup lost time	**2.31 s/veh**, **2.95 s/cycle**	2.4

6.10 Validation of the simulation against observed behaviour

We checked the simulation against the site in two ways, its discharge rate and one replayed cycle. A third check, of queue lengths, has not been done.

Discharge rate. The first simulation let a queue leave at one car every 0.72 s, against 2.31 s measured on site, and a team member watching it noticed the cars were leaving too close together. After the fix the model discharges at the measured rate within the small error given in Section 6.11.3, and an automated test keeps it there (Section 6.6).

Replay of one real cycle. We rebuilt one real 107-second cycle from the drone footage, car by car, and played it through the simulation under the real fixed-time plan. On the main through lane, cars that arrived during the green crossed the stop line within 0.37 seconds of when they did in reality, on average. The four cars already queued at the start left a few seconds early, because in the video a cluster of motorcycles sat ahead of them and the simulation has no motorcycles.

Queue lengths: not yet compared. The planned check is to run the measured arrival rates under the measured fixed-time plan and compare the resulting queues with those seen in the footage.

Table 6.11 · Observed behaviour to compare with the simulation (not yet done)

Observable	Measured on site	Status
Maximum Bx queue	12 vehicles → **≈9 cars** (see below)	not yet compared
CB queue	approximately 6 vehicles → ≈5 cars	not yet compared
CA blocked	83% of cycles	not yet compared
Ax blocked	26-29% of cycles	not yet compared
Time for Bx queue to overrun storage	approximately 5.7 minutes	not yet compared

Queue counts must be converted first. The simulation has no motorcycles, so a simulated queue is a count of cars, while the footage queues were counted as vehicles of every type; comparing them unconverted would make the model seem to under-predict by about a fifth. During the `B_morning` bin, Bx had 86 cars, 26 motorcycles and 8 heavy vehicles, so motorcycles are 22% and `cars + heavy` 78.3% of the movement.

Table 6.12 · Converting the observed Bx queue from vehicles to cars

	Vehicles	Cars + heavy
Observed maximum Bx queue	12	≈ **9.4**
Implied spacing over 78 m of storage	6.50 m per vehicle	**8.30 m per car**

The converted figure also fits the road: 6.5 m per vehicle is short for a queued car, while 8.3 m per car, with motorcycles in the gaps, is close to the usual 7-8 m queue spacing. **The simulation should be compared against about 9 queued cars, not 12.** If it reproduces the Bx spillback without being told to, that will be direct evidence it behaves like the junction. Until then it is checked on how fast queues leave and on one replayed cycle, not on how long they grow (Section 7.5).

6.11 The experiment

The batch reported here is Batch v5, run on 2 October 2026. Earlier batches were replaced, and their results appear only in Sections 6.11.2 and 6.11.3, which explain why.

6.11.1 Composition

Table 6.13 · Design of Batch v5

Scenarios	4 (MorningPeak, MiddayNormal, EveningPeak, ABACEvent)
Repetitions per scenario per controller	27
Total runs	216
Paired comparisons	108
Simulated duration per run	1,800 s after a 120 s warm-up
Controller reading interval	1.0 s
Wall-clock cost	11 h 31 min

Vehicles logged	254,917

The two runs in each pair share a random seed, so both controllers get the same vehicles, with the same driver temperaments, at the same second, and any difference in outcome comes from the control logic alone (Section 6.8).

6.11.2 Every run completed, and why the previous batch did not

All 216 runs ran the full 1,800 s, so no run is excluded and every figure in this chapter uses all 108 pairs.

The previous batch did not complete. Nineteen of its runs deadlocked, 14 under the fixed-time plan and 5 under SODA, all in the evening-peak and campus-event scenarios (a run counts as deadlocked once no vehicle leaves the junction for 120 s). In 17 of the 19 the shared kerb lane stopped first, with no Ax vehicle crossing for three to eight minutes before the whole junction froze, and none recovered.

The cause was the car model, not the traffic. A2 vehicles merging into the lane they share with Ax could drive through Ax vehicles instead of waiting for a gap: 115 of 567 test merges overlapped, some by a full car width. Two real cars cannot occupy the same space, and once enough simulated ones did, the lane jammed. The corrected model makes an A2 vehicle wait for a gap of at least 1 s (0 of 567 overlaps), and lets a vehicle that stopped short of the line at red creep up to it, most visibly on CB.

Only the car model changed. The arrival rates, the 5.4% A2 share, the 1.9 s time headway (the time gap a driver keeps behind the car ahead), the warm-up, the run length, the seeds and both controllers are identical. The corrected model is slower for the fixed-time plan, because an A2 vehicle waiting to merge now blocks lane A as it would on the road and Bx discharges about 3% fewer vehicles, but the gap between the controllers barely moved: the per-run improvement was 20.3% in the previous batch and is 20.0% in this one.

6.11.3 An earlier batch was ruled out by a car-following defect

The first full batch is not reported, because its vehicle model was wrong in a way that affected every figure. A queue in that model discharged at 0.72 s per vehicle against the 2.31 s measured here (Section 2.4), so the simulated junction could move about **3.2 times** the traffic of the real one and every waiting time was too optimistic.

Two defects caused it: every car in a stopped queue started moving in the same frame, so the start of movement never travelled back through the queue, and each car followed the one ahead at a fixed 2 m gap instead of a time gap that grows with speed. Now a following car waits until the one ahead has actually moved, and a 1.9 s time headway

sets the following distance. The corrected model discharges at 2.28 s per vehicle, within 1.3% of the field measurement, and an automated test checks the value so the defect cannot come back unnoticed (Section 6.6).

The test suite did not find this. A team member did, watching the simulation and seeing cars leave the stop line closer together than they do on site. It is the clearest case in the project of a model giving confident, believable, wrong numbers, and fixing it made the fixed-time plan harder to beat, since its own waits rose once the simulated junction lost its extra capacity.

6.12 Main result: waiting time per vehicle

The reported average is taken over vehicles, not over runs. Every vehicle in an analysed run counts once, so a run carrying about 1,813 vehicles weighs more than one carrying about 711. That is what a road user experiences. Averaging each run's mean instead gives every run equal weight however many vehicles it served; we report that too, because the standard paired tests work on per-run values, and Section 6.12.2 explains why the two differ.

6.12.1 Result

Table 6.14 · Mean wait per vehicle over all pairs, paired bootstrap (SODA − fixed)

	Fixed-time plan	SODA	Difference
Mean wait per vehicle	**44.15 s**	**38.44 s**	**−5.70 s (−12.9%)**
Vehicles contributing	125,490	129,427	
95% CI of the difference (paired bootstrap)			[−8.12, −3.39] s
Significance (paired bootstrap)			$p < 0.0001$

The paired bootstrap resamples the 108 pairs with replacement 10,000 times and recomputes both per-vehicle means each time; the spread of the difference gives the interval and p. Resampling whole pairs keeps the pairing and each run's vehicles together.

The same comparison per run, which is the form the standard paired tests take:

Table 6.15 · The same comparison on per-run means, with paired tests

	Fixed-time plan	SODA	Difference
Mean of run means	42.03 s	33.62 s	−8.40 s (−20.0%)
95% CI of the paired difference			[−11.10, −5.71] s
Paired t-test			$t(107) = -6.18$, $p < 10^{-6}$
Wilcoxon signed-rank			$p < 10^{-6}$
Cohen's d (paired)			0.59

The paired differences are not normally distributed (Shapiro-Wilk $p = 0.005$), so the Wilcoxon test is the one to rely on; the two tests agree.

6.12.2 Why the two averages differ

The per-run figure (−20.0%) is larger than the per-vehicle figure (−12.9%) because runs are not the same size. A campus event run serves about 1,813 vehicles and a midday run about 711, yet each counts once in the per-run mean, so the per-run figure over-weights the quiet scenarios.

Table 6.16 · Per-vehicle and per-run change by scenario

Scenario	Vehicles	Per-vehicle change	Per-run change
Morning peak	21,595	−65.7%	−65.8%
Midday	19,185	−40.5%	−40.3%
Evening peak	35,748	+6.8%	+6.3%
Campus event	48,962	−3.2%	−0.8%

The scenarios where SODA does worst carry the most vehicles, so weighting by vehicle gives the smaller, more cautious figure. **It is the one this report leads with.**

6.13 Worst-case waiting time

Chapter 5 showed the problem here is the few vehicles that wait far longer than the rest: Bx runs at v/c 1.51 while A runs at 0.25. A controller that lowers the mean but stretches those waits has not fixed it, so the worst-case wait, the longest any single vehicle waited in a run, is a main result.

Table 6.17 · Worst-case wait per run

	Fixed-time plan	SODA	Difference
Mean of per-run maximum wait	**204.2 s**	**119.3 s**	**−84.9 s (−41.6%)**
95% CI of the paired difference			[−95.3, −74.6] s
Paired *t*-test			$t(107) = -16.24$, $p < 10^{-6}$
Wilcoxon signed-rank			$p < 10^{-6}$
Cohen's *d* (paired)			1.56

The worst-served vehicle in a run waited 84.9 s less under SODA. The effect is large ($d = 1.56$), larger than its effect on the mean ($d = 0.59$): the tail is where the controller makes the most difference. The differences are not normally distributed (Shapiro-Wilk $p = 0.001$), so the Wilcoxon result is the one to rely on.

Runs in which SODA's maximum wait still exceeded 60 s: **55 of 108**. The anti-starvation rule bounds how long a lane is ignored, not how long a vehicle waits, so a long tail survives on lanes that never empty (Section 7.4).

6.14 Throughput

Table 6.18 · Vehicles cleared per run

	Fixed-time plan	SODA	Difference
Vehicles cleared per run	1,162	1,198	+36 (+3.1%)
Paired test			$t(107) = 1.90$, $p = 0.060$; Wilcoxon $p = 1 \times 10^{-5}$

SODA cleared 36 more vehicles per run (+3.1%, Wilcoxon $p = 1 \times 10^{-5}$). The paired t-test gives $p = 0.060$, but the differences are not normally distributed (Shapiro-Wilk $p < 10^{-6}$), so by the rule in Section 6.8 the Wilcoxon result is the one to rely on.

We report throughput because a controller can lower mean delay by serving fewer vehicles. That did not happen here: delay fell while throughput rose.

6.15 Results by scenario

Table 6.19 · Mean wait per vehicle by scenario

Scenario	Pairs	Fixed	SODA	Change	95% CI of difference	p (bootstrap)
Morning peak	27	40.16 s	13.77 s	−65.7%	[−27.6, −25.1] s	$p < 0.0001$
Midday	27	25.71 s	15.28 s	−40.5%	[−11.1, −9.8] s	$p < 0.0001$
Evening peak	27	54.49 s	58.17 s	+6.8%	[0.5, 6.7] s	$p = 0.024$
Campus event	27	45.58 s	44.14 s	−3.2%	[−4.9, 2.4] s	$p = 0.418$

Mean wait per vehicle, the same measure as Figure 6.3. The difference is SODA minus fixed-time, so an interval above zero means SODA was slower. Per-run means for each scenario are in Section 6.12.2.

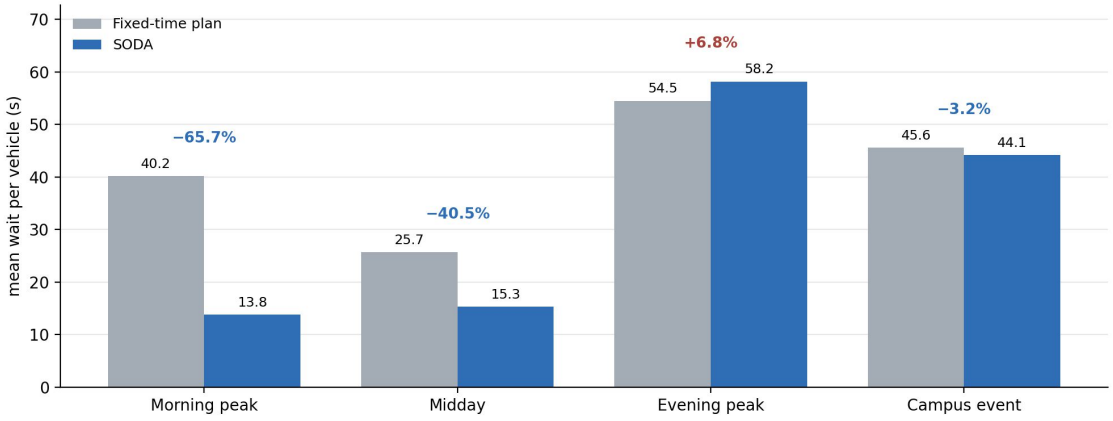


Figure 6.3. Mean wait per vehicle by scenario, fixed-time plan against SODA, over every pair analysed. The improvement is largest where some approaches are often empty, and it disappears when every approach is saturated at the same time.

The result differs from scenario to scenario, and the differences follow a consistent pattern. Ranked by the change in wait per vehicle: morning peak (−65.7%), midday (−40.5%), campus event (−3.2%, not significant), evening peak (+6.8%). The gain is largest where some approaches are often empty and reverses in the evening peak scenario, where every approach saturates at once.

6.15.1 Where the gain comes from

SODA has two advantages over a fixed-time plan, and both need spare capacity:

1. It skips empty movements. A fixed-time plan serves a movement whether or not anyone is waiting. SODA skipped 214 signal heads per run in the morning peak scenario.
2. It ends a green once its queue has cleared. The gap-out rule (Section 6.3) passes unused green to the next movement. In the midday scenario, 45% of SODA's phases ended by gap-out, returning green that a fixed plan would have run to the end.

6.15.2 Where it loses, and why

The evening peak scenario is the condition in which SODA is worse than the fixed plan (+6.8%, $p = 0.024$). The paired t-test on per-run means, which weights every run equally, gives $p = 0.081$ for the same comparison; the per-vehicle bootstrap is the test of the measure this report leads with. It is the reverse of Section 6.15.1: when every approach is full at the same time, both advantages disappear.

Table 6.20 · Evening peak: the switching overhead behind SODA's loss

Per run	Fixed-time plan	SODA
Signal heads skipped for being empty	0	5
Phases ending by gap-out	none	1%

Per run	Fixed-time plan	SODA
Phases	51	71
Time spent in amber and all-red	255 s	396 s
Start-up lost time	150 s	208 s
Capacity consumed by switching		**11.1% of capacity (200 s per run)**

With no movement ever emptying, no green ends early, and with every movement waiting, some movement always passes the 40.5 s anti-starvation threshold. So 93% of greens are cut at the earliest moment the rule allows, 20 s in, and SODA turns into a fixed rotation that pays amber, all-red and startup lost time at every switch: 11.1% of capacity (200 s per run) that the fixed plan does not spend.

What it buys with that capacity is shorter worst-case waits. Even in the evening peak scenario, wait on Bx went from 163.5 s to 103.5 s (−36.7%) and the worst-case wait from 233.2 s to 194.2 s (−16.7%). The time comes out of the through movements A and B, which carry 4.3 times as many vehicles as Bx, so the average across all vehicles rises.

The parameter `minGreenBeforeInterrupt` therefore sets the trade between mean delay and fairness under saturation. We held it at 20 s, as the specification sets it, and did not tune it against these results; Section 8.3 proposes a sensitivity study.

6.16 Results by movement: did the controller find the fix?

Chapter 5 concluded from field measurement alone that green should move from the over-served through movement A to the starved right turn Bx. The controller was not told this: it gets four vehicle counts a second and knows nothing of v/c ratios, queue storage or the guard at the gate.

Table 6.21 · Mean wait by movement

Movement	Fixed-time plan	SODA	Change	Vehicles cleared
A	66.3 s	83.6 s	+26.2%	16,582 → 14,290
A2	24.3 s	33.7 s	+38.4%	1,187 → 1,151
Ax	20.1 s	19.0 s	−5.5%	30,772 → 29,310
B	24.2 s	28.3 s	+17.0%	28,483 → 30,773
Bx	137.8 s	63.5 s	−53.9%	14,386 → 18,146
CB	63.5 s	62.6 s	−1.4%	13,905 → 14,738
CA	11.8 s	11.3 s	−4.0%	20,175 → 21,019

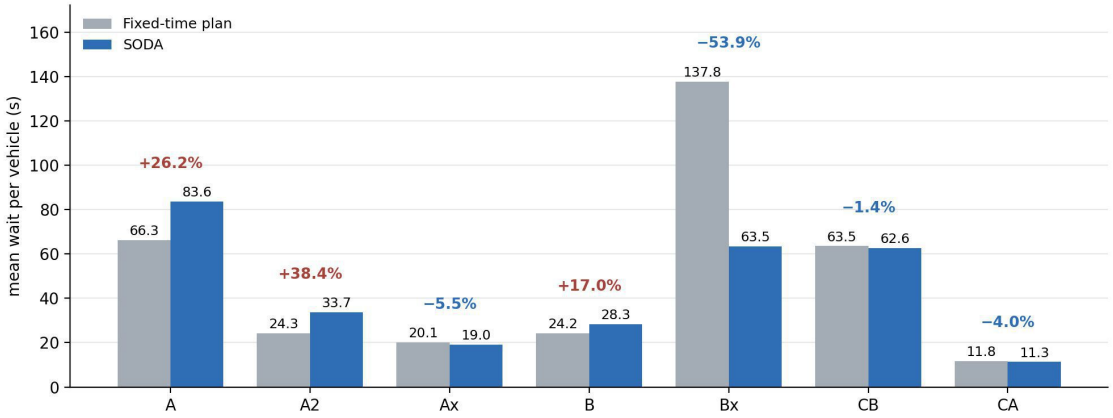


Figure 6.4. Mean wait per vehicle by movement. The controller moved green from the through movement A to the starved right turn Bx without being told to. The unsignalised movements Ax and CA change only through the movements that block them.

Wait on Bx went from 137.8 s to 63.5 s (−53.9%), and SODA cleared 26% more Bx vehicles. Paired bootstrap over pairs: 95% CI of the difference [−81.2, −67.2] s, $p < 0.0001$.

This is the main claim of the project, and the field work predicted it: green moved away from A, whose wait changed by +26.2%, and towards Bx, whose wait changed by −53.9%. The controller reached the reallocation Chapter 5 derived from v/c ratios using nothing but four vehicle counts per second.

6.16.1 The cost falls on the movements that could afford it

Bx carries 11.5% of vehicles, but its change is large enough that it alone moves the overall mean by −8.51 s, while the change on A moves it by +2.30 s. The percentages in the table hide this: a large change on a small movement can outweigh a smaller one on a busy movement.

Table 6.22 · Each movement's contribution to the change in overall mean wait

Movement	Change in mean wait	Share of all vehicles	Contribution to the overall mean
Bx	−74.3 s	11.5%	−8.51 s
A	+17.4 s	13.2%	+2.30 s
B	+4.1 s	22.7%	+0.93 s
Ax	−1.1 s	24.5%	−0.27 s

6.16.2 The unsignalised movements

Ax and CA have no signal head, so any change in their waiting time comes indirectly, from clearing the movements that block them (Section 5.6).

Table 6.23 · The unsignalised movements, helped only indirectly

Movement	Blocked by	Fixed-time plan	SODA	Change
Ax	A2 in the shared kerb lane; Bx, which it must yield to	20.1 s	19.0 s	−5.5%
CA	the CB queue in the shared exit	11.8 s	11.3 s	−4.0%

Ax changed by −5.5% and CA by −4.0%. Both changed little, which is what to expect of movements that can only be helped by clearing what blocks them.

A validation script once checked that these two recorded about zero delay, on the reasoning that a movement with no red light has nothing to wait for. **That check was wrong, for the reason in Section 5.6:** the free movements here share lanes with movements that do stop. We removed the check rather than bend the simulation to pass it.

6.17 Summary of the simulation result

KEY RESULT

Table 6.24 · Summary of the simulation result, with the test used for each measure

Measure	Fixed-time plan	SODA	Change	Significance
Mean wait per vehicle	44.15 s	38.44 s	**−12.9%**	$p < 0.0001$ (bootstrap)
Worst-case wait	204.2 s	119.3 s	**−41.6%**	$p < 10^{-6}$ (Wilcoxon)
Wait on Bx, the starved movement	137.8 s	63.5 s	**−53.9%**	$p < 0.0001$ (bootstrap)
Vehicles cleared	1,162	1,198	**+3.1%**	$p = 1 \times 10^{-5}$ (Wilcoxon)

SODA changed the average wait per vehicle by −12.9% and the worst-case wait by −41.6%, and it did so largely by finding the reallocation the field measurements prescribed: Bx, the movement at v/c 1.51, changed by −53.9%. The gain is concentrated where the junction has slack and reverses in the evening peak scenario, where the anti-starvation rule turns the controller into a fixed rotation and switching costs about as much as it saves.

This is a simulation result. SODA has never run a signal on a public road, and Chapter 7 sets out when these figures would not carry over to the real junction.

7 · Limitations

> **IN SHORT**
> The field results hold for dry daylight and the sessions we measured. The controller results hold in simulation only: the simulation has no motorcycles, and two of its four scenarios use partly or wholly estimated demand.

7.1 What the field data does not cover

No wet weather. We collected the data in the rainy season. It rained about three days in seven and drones cannot fly in rain, so we lost close to half the available days, and wind spoiled the framing on most of the rest; the Lane C evening session planned for Friday 21 August 2026 was rained off to the Monday. So the dataset has no wet-weather images, though a system installed here would often meet rain. This could not be avoided, because the weather that brings rain is the weather that grounds the drone, and augmentation cannot fake believable rain. The detector is limited to dry conditions and untested in rain. A mast-mounted camera, the intended sensor anyway, could collect rain footage because it does not need to fly.

The A approach was never measured at true peak. `A_morning` was recorded on a Monday, which the guard described as a low-volume day, so it understates demand on A and Ax. We apply no correction: the ×0.7 weekday multiplier is only a spoken estimate, and we would rather report a known gap than rest a headline figure on a guess. `B_morning` was recorded on a Tuesday, a real peak day, so only approach A is affected. The main pattern holds either way, since the Ax:A ratio of about 3.5:1 is the same on a Monday morning and a Wednesday midday across a doubling of volume; only the size of demand at true peak is missing.

The detector is least accurate in light traffic. Over-counting runs from +1.7% at 298 vehicles to +17.2% at 128 (Section 4.4), strictly in order only within one approach. The peak measurements the capacity analysis rests on are the most reliable; off-peak figures carry clearly more error.

Saturation headway was measured on one movement. ℓ_1 = 2.95 s and h = 2.31 s come from five green phases on the B approach's through movement in the morning peak, and we apply them to the turns Ax, Bx and CB as well. Turning traffic should discharge more slowly, so the true headway on Bx and CB is probably higher and their green needs and v/c ratios are understated here. The error is on the safe side: correcting it would make Bx and CB look worse and strengthen the central finding. Timing a turning movement would be a small addition to any future session.

7.2 Heavy vehicles and platform drift

Chapter 4 covers both in full. Heavy vehicles cannot be reported separately because the hand counts and the detector define the class differently (Section 4.5); vehicle counts and `cars + heavy` are unaffected, and only PCU flow from the hand counts is overstated, by 13.7% overall. Platform drift of 9-92 px does not change counts on a wide ROI (Section 4.6), but that test covers one session, and drift in `A_morning` itself cannot be measured because the striped barrier in that view defeats patch matching.

7.3 What the simulation leaves out

Motorcycles. They are 21% of annotated instances and 22-35% of individual movements, but the simulation spawns none; it runs cars only, on the cars + heavy rates of Section 6.9. Other chapters refer back to this reasoning. Motorcycles do not lengthen queues: they filter to the front and take almost no storage, so spawning them as queueing agents would produce queues far longer than observed. They do use discharge time, but that is already inside the measured headway: h = 2.31 s is higher than the HCM's 2.00 s precisely because filtered motorcycles must clear the stop line before the cars behind them move. Modelling them as agents on top of a headway measured on mixed traffic would count the same effect twice. Two real limits follow. The approach assumes a roughly even motorcycle share across movements, and it is not even (22% of Bx against 35% of Ax at midday), so a movement with a high share has its discharge time slightly understated. And simulated queues count cars while observed ones counted all vehicles, so Section 6.10 must convert before comparing.

The neighbouring four-way junction. At peak the Bx queue reaches back into the four-way junction 78 m upstream, which lies outside the simulation. The Bx delay shown is therefore a lower bound: the model captures neither the delay caused at the four-way nor any congestion feeding back from it.

Layout that changes during the day. The Lane C exit closes for 15-30 minutes between 08:30 and 09:00, and another way out opens between 15:30 and 17:00. The simulation uses one fixed layout.

Manual guard intervention. By the operator's estimate, guards wave left-turning vehicles in 40-60% of the time, but we never turned this into a rate: at the drone's height, a guard waving traffic through cannot be told apart reliably from one standing near the median. The model holds the whole queue where the guard releases part of it, so the reported Ax delay is an upper bound. The observation matters more than the missing rate: the junction works partly because people step in by hand.

Estimated demand in two scenarios. Morning and midday use measured rates, apart from the morning exit movements, which were never measured (Section 6.9). The evening peak is partly estimated and the campus event wholly estimated, its rates scaled down after the first version carried more traffic than the junction could physically handle. Results for those two describe a believable junction more than a measured one.

Rates from different days. The drone filmed each approach on a different day (A on a Monday, B on a Tuesday), and the scenarios combine these rates as if they happened together.

Perfect counts. SODA reads exact counts in the simulation. A real camera misses some vehicles (Chapter 4), and one missed vehicle can shift a queue clock by one reading (1 s).

7.4 What the controller cannot reach

The busiest movement has no signal. Ax carries the largest share of demand, and CA has the same problem at the exit. No signal controller can act on either directly; the only lever is clearing the signalised movement that blocks them, faster.

The fairness rule works in lane time, while delay is measured per vehicle. The override moves to the front any head that has waited 42.75 s, 0.95 × the 45 s maxWaitTime taken from Level of Service (LOS) D, the HCM's letter grade for how well a junction serves traffic. It works on queueStart, the moment a head last went from empty to occupied, which is not when any particular vehicle arrived. A head that never empties restarts its clock only when served, so a vehicle joining an already occupied lane inherits a clock that may have started long before it. The measure we report is each vehicle's own wait, logged as it leaves its region, and on a lane that never empties the two can drift apart without limit.

An earlier batch showed this gap directly and exposed a separate fault: the queueStart reset applied only to heads leaving green, so a head that stayed green across phases kept its clock for ever. Fixing that narrowed the gap without closing it. In the reported batch 55 of 108 adaptive runs still recorded a maximum per-vehicle wait above 60 s against the 45 s setting, because the rule still tracks whether a lane is occupied, not how long any one vehicle has waited. Closing the gap needs an arrival time for each vehicle, which a sampled camera detector does not provide (Section 6.1). SODA guarantees that no lane is ignored for long, not that every vehicle is served within a set time.

No link to real signals. We built no interface to real signal hardware, and the detector has not been tested on roadside hardware, in rain or at night.

7.5 Validation still to be done

The simulation's queues discharge at the measured rate (Section 6.11.3), and a replayed real cycle matches observed stop-line crossings on the main through lane to within 0.37 s on average (Section 6.10). The queue-length comparison planned in Section 6.10 has not been made, so simulated queue lengths are not yet checked against observed ones.

8 · Conclusions and future work

> IN SHORT
> **Recommendation:** move 10 s of green from A to Bx. It needs no new equipment and no change to the cycle. A drone camera counts traffic accurately enough to show the problem (within 1.7%), and in simulation SODA found the same fix by itself.

8.1 What we found

The signal shares out green almost the opposite way to demand. Both movements serving the university are at or beyond capacity (Bx entering at v/c 1.51, CB exiting at 1.04), while both through movements hold more than three times the green they need (v/c 0.25). Moving ten seconds of green from A to Bx brings Bx to v/c 0.95 and clears its queue; A still holds twice what it needs, and the cycle, phases and layout stay the same. Measuring on site changed the answer: the measured 2.95 s startup lost time raised Bx from v/c 1.28 to 1.51 and the fix from eight seconds to ten.

The junction has one repeating type of failure. Wherever an unsignalised movement shares a lane with a signalised one, a signal that does not apply to it holds it back: in 26-29% of cycles at the entry and 83% at the exit. On approach A the unsignalised left turn outnumbers through traffic 3.5 to 1.

A camera counts accurately enough to be the measuring tool. Total counts are within 1.7% of the hand counts, and the `cars + heavy` counts, the group the simulation models, within 2.7%.

In simulation, SODA found the fix the field measurements pointed to. Across 108 pairs of runs, mean wait per vehicle changed by −12.9%, worst-case wait by −41.6% and wait on the starved Bx movement by −53.9%, with SODA moving green from A to Bx on its own. It does not win everywhere: when every approach is full at once it loses capacity to switching, and its average wait in the evening peak scenario was 6.8% higher than the fixed-time plan's ($p = 0.024$) (Section 6.15.2). The simulation reproduces the measured discharge rate and one replayed real cycle; its queue lengths have not yet been compared with those observed (Section 6.10).

8.2 Recommendation

The first recommendation can be applied now: the ten-second change of green in Section 5.4. It needs no capital spending, no change to the layout and no new equipment.

The longer-term one is about the road, and signal timing cannot deliver it. The shared lane's volume-to-capacity ratio is only 0.56, yet blocking still happens, so the problem on Ax is the layout, not capacity. A dedicated left-turn lane would fix it; signal timing can only ease it.

8.3 Future work

Data. Collect wet-weather footage from a fixed mast camera, which has none of the flight limits that stopped us. Measure approach A at its true peak, Tuesday to Thursday between 08:15 and 09:00, the window this project found (the collection guide had assumed 07:00-08:30). Turn guard intervention into a rate, so the largest factor left out of the model becomes a number. Re-count one session under the three-class definition to settle the heavy class.

Model. Add the neighbouring four-way junction, so the Bx delay becomes an estimate instead of a lower bound. Test a lightweight detector on pole-mounted hardware to show whether the system can run at the junction rather than on a workstation. Compare simulated queue lengths with observed ones (Section 6.10). Build and test an interface to a real signal controller.

Controller. Two changes target the capacity lost to switching, which caused the evening result.

- *Sensitivity of* `minGreenBeforeInterrupt`. We held it at 20 s, as specified. Testing a range would show how mean delay trades against worst-case wait under saturation.
- *Per-head gap-out.* A green now ends early only when the whole group reads empty, so a head that clears first holds idle green with nothing to serve. A first step, about fifteen lines with no extra clearance, ends the phase once every head has read empty twice in a row, even if they emptied at different times. It recovers only a small share of the idle green, because in most phases at least one head never empties. Recovering the rest means switching a single head to red mid-phase, with its own amber, all-red and conflict check, which is a specification change; it would work per signal head, not per lane, since A and A2 share one head. The 1 s reading interval draws on the same idle green, so the two savings do not add up.

9 · References

Department of Highways, Thailand (2019). *Traffic Engineering Design Standard.* Ministry of Transport, Bangkok.

Du, D. et al. (2019). *VisDrone-DET2019: The Vision Meets Drone Object Detection in Image Challenge Results.* ICCV Workshops.

Google Thailand / DeepMind (2023). *Using AI to reduce traffic congestion in cities.* Google Blog.

Hunt, P.B., Robertson, D.I., Bretherton, R.D. and Royle, M.C. (1982). *SCOOT — A Traffic Responsive Method of Coordinating Signals.* TRRL Laboratory Report 1014.

Jocher, G., Chaurasia, A. and Qiu, J. (2023). *Ultralytics YOLOv8.* github.com/ultralytics/ultralytics

Lowrie, P.R. (1990). *SCATS: The Sydney Coordinated Adaptive Traffic System.* Roads and Traffic Authority of New South Wales.

Transportation Research Board (2022). *Highway Capacity Manual, 7th Edition.* National Academies of Sciences, Engineering, and Medicine.

Webster, F.V. (1958). *Traffic Signal Settings.* Road Research Technical Paper No. 39, HMSO, London.

Zhang, Y. et al. (2022). *ByteTrack: Multi-Object Tracking by Associating Every Detection Box.* ECCV.

Appendix A · Arrival rates

A.1 Why the simulation uses the hand-count figures, not detector output

We considered a second set of rates, A = 19.7, Ax = 5.6, B = 8.7, made by applying the Section 4.4 correction factor to detector totals and splitting by movement. **We rejected it for three reasons.**

First, it is not independent. The detector cannot separate A from Ax, since every ROI file for approach A has one polygon covering both, so the split has to come from the hand count anyway.

B and Bx use separate lanes, so we have since measured them separately: a split run over the same 900 s reproduces the hand-counted 52:48 division exactly, at a combined correction factor of 0.890x, the same as the single-region factor, which confirms the

hand-count split. A and Ax share one lane (Section 5.6) and cannot be split. Applying a factor with a 9.4% spread (Section 4.4) and then a hand-counted ratio adds two layers of error to a number the hand count already gives directly.

Second, every hand-count figure can be rebuilt from the tally files in one step:

Table A.1 · Arrival intervals from the hand counts, as used in the simulation

Movement	cars + heavy in 900 s	900 ÷ count	Used
A	35 + 13 = 48	18.75 s	**18.8**
Ax	154 + 24 = 178	5.06 s	**5.1**
B	67 + 33 = 100	9.00 s	**9.0**
Bx	86 + 8 = 94	9.57 s	**9.6**

Third, Chapter 5's v/c table is built on the same figures, and feeding the simulation a different set would leave Chapters 5 and 6 describing two different junctions.

The main result does not depend on the choice: the two sets agree on Bx within 0.3% (9.57 against 9.60), and the Bx oversaturation finding rests on Bx alone.

They differ most at midday, where `B_midday` has the largest detector over-count (+17.2%, Section 4.4): its detector-based figures of 15.9 s and 17.6 s, against the hand count's 20.5 s and 21.4 s, would have loaded the simulation about 25% heavier than the traffic we observed.

Appendix B · Dataset settings

B.1 Footage and frame extraction

20 session folders cover approaches A, B and C morning, midday and evening, recorded July-August 2026: about 6.6 hours at 1920 × 1080 and 29.97 fps, HEVC. Counting flights hovered about 6 m up; one survey flight flew at 90 m.

Raw clips are stored in `DATA/DATA_LANE_<A|B|C>_<MORNING|MIDDAY|EVENING>/` and `DATA/DATA_OVERHEAD_DRONE/`. The full set is backed up on Google Drive.

`yolo_pipeline/extract_frames.py` takes one frame every 3.5 s (the default `--interval`) into `frames/<session>/`, for one video (`--video`) or many (`--batch`).

B.2 Annotation

Roboflow workspace `abactraffic`, project `abac-vehicle-detection`. The final model uses dataset version v2, generated on 2 September 2026 at 15:13: two annotators, 6,585 images, 57,260 instances, exported in YOLOv8 format.

The classes are the three listed in Section 3.2. The first version of the dataset had five classes (car, motorcycle, truck, bus, special type) and 160 heavy instances.

B.3 Split assignment

Every session folder is in exactly one split.

Table B.1 · Images in each split

Split	Sessions	Images
Train	12	4,075 (62%)
Validation	4	1,406 (21%)
Test	4	1,104 (17%)

The test sessions are `A_midday_2`, `B_midday_2`, `B_evening_aug14_3` and `C_midday_2`.

Step 3 of `train_v2.ipynb` maps every exported filename back to its session and stops if any session appears in two splits.

B.4 Preprocessing and augmentation

Auto-orient, then fit within 1280 × 1280 keeping the aspect ratio (1920 × 1080 becomes 1280 × 720).

A 2× multiplier gave 8,150 training images from 4,075; the generated version holds 10,660 images overall.

Table B.2 · Full augmentation settings, dataset version v2

Augmentation	Setting
Flip	horizontal + vertical
Rotation	±5°
Brightness	±20%
Exposure	±10%
Blur	0.2 px
Noise	0.5%

Shear, heavy geometric distortion, large hue shifts, greyscale and auto-contrast were left out on purpose. The first version used a 3× multiplier, rotation ±15% and noise 1.01%. Section 3.4 gives the reasons.

Appendix C · Detector calibration

C.1 Over-count against traffic volume

One explanation we considered for the Section 4.4 over-count was that ByteTrack drops a vehicle that stands in a queue too long and picks it up again under a new ID. The data rule this out: `B_midday` has no queues at all, yet the worst over-count. The over-count follows traffic volume instead:

Table C.1 · Detector over-count against traffic volume, per session

Session	Approach	Manual total	Over-count
`B_midday`	B	128	+17.2%
`A_midday`	A	181	+5.5%
`B_morning`	B	253	+11.9%
`C_evening`	C	259	+8.1%
`A_morning`	A	298	+1.7%

The over-count falls as volume rises, but not cleanly (correlation r = −0.69). **Within one approach the pattern is strict**: on A it falls from 5.5% to 1.7% as volume rises from 181 to 298, on B from 17.2% to 11.9% as volume rises from 128 to 253. But B sits about ten points above A at similar volume, a gap the data do not explain; B is filmed at a shallower angle, a believable but untested cause.

The first model's four-session table showed a clean ordering that did not survive adding a third approach, and claiming it would be more than five points can support. We keep the rejected explanation here because a believable mechanism the data contradict is worth knowing about.

C.2 The heavy class: evidence for the definition mismatch

We expected four times as many heavy examples to either stabilise the class or trace the problem to resolution. Neither happened: the session-level factor got worse (Section 4.4) while per-instance detection got better (Section 3.7), and one failure cannot cause both.

Per instance, heavy is detected as well as the other classes: over-detected by 5.3% (factor 0.950×), within a 2.9% band of car (0.967×) and motorcycle (0.939×), with only 38 of 14,493 validation instances (0.26%) given the wrong class.

The missing heavy vehicles reappear as cars, with some to spare: across all five sessions the detector finds 128 fewer heavy vehicles than the hand count and 181 more cars, 141% of the shortfall. The excess over 100% is the ordinary false-positive rate of Section 4.4.

The detector agrees with its own training set, not with the hand count.

Table C.2 · Heavy-vehicle share in the hand count, the annotation set and the detector

Source	Heavy share of all vehicles
Manual tally, five sessions	**13.6%** (152 of 1,119)
Annotation set, v2 (Section 3.2)	**1.1%** (656 of 57,260)
Detector, five sessions	**2.0%** (24 of 1,207)

A detector failing at a class would disagree with its annotation set about how common it is. This one agrees within about one point and differs from the hand count tenfold: the disagreement is between the hand count and the annotation set, created by the taxonomy decision of Section 3.2. The hand count overstates PCU in each session as follows.

Table C.3 · PCU flow as tallied and as corrected, per session

Session	PCU as tallied	PCU corrected	Overstated by
`A_morning`	286.8	256.8	11.7%
`A_midday`	171.8	142.8	20.3%
`B_morning`	254.5	221.5	14.9%
`B_midday`	126.9	101.9	24.5%
`C_evening`	220.1	209.1	5.3%
All five	**1,060.0**	**932.0**	**13.7%**

"Corrected" moves each session's extra hand-counted heavy vehicles to `car` at PCU 1.0.

C.3 ROI depth: detections by image row and the clipping test

Single-frame inference on 29 frames per clip, every detection inside the ROI grouped by image row:

Table C.4 · Detections by image row, B_morning: far boxes are small and uncertain

`B_morning`, image row	n	Median box height	Median confidence
0-200 (far)	39	**41 px**	0.64
200-300	39	58 px	0.70
300-400	28	88 px	0.72
400-452	10	115 px	0.76
452-600	26	143 px	0.77
600-800	25	214 px	0.77
800-1080 (near)	21	**280 px**	0.71

All five sessions recounted with the ROI clipped where it narrows to half its widest, keeping 61-66% of its depth:

Table C.5 · Correction factors at full and clipped ROI depth

Arm	A_morn	A_mid	B_morn	B_mid	C_eve	Mean	Spread
Full depth (reported)	0.974×	0.969×	0.890×	0.887×	0.956×	0.935×	**9.4%**
Clipped	0.991×	1.016×	0.975×	1.000×	**1.081×**	1.013×	**10.5%**

On the four free-flowing approaches the mean is 0.996× at 4.1% spread; the two worst sessions move from 0.890× and 0.887× to 0.975× and 1.000×, and `B_midday` lands on exactly 86 against 86. Hand-drawn clipped movement polygons for the same session give 0.989×, so two differently built clipped ROIs agree within one vehicle. `C_evening` breaks the pattern, overshooting to 1.081× (an 8% under-count) and pushing the spread to 10.5%.

Two explanations were tested and rejected first. One was that `C_evening`'s ROI never reaches small-vehicle range, so a relative clip cuts too much; the reverse is true, since it is the most distant framing of the five (32 px in the clipped region, 87 px in the region kept). The other was a window-edge effect, a vehicle inside the 0-900 s window still in transit when it closes; recounting to 960 s recovered three vehicles, not thirteen.

The cause is the standing queue at the university gate, with about half the vehicles stopped above the cut (Section 4.7), which no longer window can recover.

Appendix D · Controller implementation

D.1 Green-time calculation

The green-time calculation in `AdaptiveController.cs`:

```
float CalculateGreenDuration(List<int> group)
{
    float maxNeeded = gMin;
    foreach (int s in group)
    {
        float d = DemandOf(s);
        if (d <= 0f) continue;
        float needed = startupLostTime + d * saturationHeadway;
        if (needed > maxNeeded) maxNeeded = needed;
    }
    return Mathf.Clamp(maxNeeded, gMin, gMax);
}
```

D.2 Why the proposal's share formula could not be tuned

`G_min` cannot be tuned around the failure described in Section 6.2. **Lowering it makes Bx worse.** For any movement whose share is below 50%, the floor removed is larger than the extra range gained:

Table D.1 · What Bx receives as the proposal formula's G_min is lowered

`G_min`	Bx receives
8 s	20.9 s
6 s	19.6 s
4 s	18.3 s
2 s	17.0 s

Raising `G_max` helps only crudely: Bx reaches 29.6 s at `G_max = 70 s`, an unrealistically long green for a single-lane turn that would also distort every other demand pattern.

D.3 Where the bounds bind

The table shows the raw green for each demand D and the green after the clamp.

Table D.2 · Raw and clamped green for each demand

Demand D	G_raw	G after clamp
0-2	2.95-7.57	**8.00** (floor)
3	9.88	9.88
8	21.43	21.43

Demand D	G_raw	G after clamp
15	37.60	37.60
18	44.53	44.53
19+	46.84+	**45.00** (ceiling)

D.4 Reading-interval test

The unchanged controller (`soda.py`) was driven from a point-queue simulation (vehicles wait at a point, with no length or car-following) on the Section 6.9 inputs: MorningPeak and MiddayNormal rates, 120 s warm-up, 1,800 s measured, 30 paired seeds per setting, and a detector that misses each vehicle on 1.7% of readings. Differences are paired against 2 s; negative means shorter waits. A perfect detector gives the same ordering (`sample_interval_results.md`). With no motorcycles or shared-lane blocking, the model ranks the settings but its waits are not comparable with Unity's.

Table D.3 · Point-queue test of the reading interval

Scenario	Reading every	n	Mean wait	Δ vs 2 s [95% CI]	Paired t (p)	Wilcoxon W (p)	Cohen's d
MorningPeak	0.5 s	30	10.85 s	−0.63 s [−0.93, −0.34]	t(29) = −4.35 (1.5e-04)	56 (1.1e-04)	−0.80
MorningPeak	**1 s (adopted)**	30	11.07 s	−0.42 s [−0.69, −0.15]	t(29) = −3.16 (0.004)	97 (0.004)	−0.58
MorningPeak	2 s (original)	30	11.48 s	baseline	—	—	—
MorningPeak	3 s	30	11.55 s	+0.07 s [−0.23, +0.37]	t(29) = 0.47 (0.640)	212 (0.685)	0.09
MiddayNormal	0.5 s	30	18.21 s	−2.67 s [−3.12, −2.21]	t(29) = −11.94 (1.0e-12)	0 (1.9e-09)	−2.18
MiddayNormal	**1 s (adopted)**	30	19.28 s	−1.60 s [−1.94, −1.26]	t(29) = −9.68 (1.4e-10)	3 (9.3e-09)	−1.77
MiddayNormal	2 s (original)	30	20.88 s	baseline	—	—	—
MiddayNormal	3 s	30	21.61 s	+0.73 s [+0.23, +1.23]	t(29) = 3.01 (0.005)	98 (0.005)	0.55

0.5 s looks better again in the table, but only because the model treats each missed detection as independent (Section 6.1).

D.5 Simulation output files

Each run writes two CSV files (plain spreadsheet-style text files) and a phase log:

```
detailed_<controller>_<scenario>_<timestamp>.csv
  ControllerType, Scenario, Timestamp, Lane, DriverType, WaitTime

summary_all_runs.csv                    (appended across runs)
  ControllerType, Scenario, RunTimestamp, TotalSpawned,
  VehiclesCleared, AvgWaitTime, MaxWaitTime, StarvationEvents
```

Every run also writes a per-phase file, `phases_<controller>_<scenario>_<timestamp>.csv`. The summary file carries further columns, such as skipped heads and idle lane time, which Section 6.15 uses.

`Lane` records each vehicle's movement, so Ax and A2 stay separate even though they share one ROI object. `StarvationEvents` counts anti-starvation events, which exposes a controller that lowers mean delay by neglecting a light movement. One starving head logs two starvation events, the interrupt and then the override, so `StarvationEvents` counts each episode twice.

Appendix E · Training environment

E.1 Environment

Every stage ran in Google Colab on a requested NVIDIA T4 GPU (`Runtime → Change runtime type → T4 GPU`); each notebook prints `nvidia-smi` and `torch.cuda.get_device_name(0)` at the top. Weights go to Google Drive at `/content/drive/MyDrive/SP1_ABAC_Traffic/` . `save_period=10` writes a checkpoint every ten epochs so a disconnect loses at most ten, and step 5 resumes from `last.pt` .

E.2 The four training stages

Each stage starts from the best weights (`best.pt`) of the stage before it.

Table E.1 · The four training stages

Stage	Notebook	Data	Settings	Output and result
1 · COCO	none (`yolov8s.pt` downloaded from Ultralytics, not trained by us)	80 classes, about 330k street-level photos	—	`yolov8s.pt`
2 · VisDrone	`yolo_pipeline/pretrain_colab.ipynb`	Roboflow Universe, workspace `udy` , project `visdrone-qr0vn` , version 2	40 epochs, image size 640, batch 16, patience 10	`overhead_pretrain.pt`

Stage	Notebook	Data	Settings	Output and result
3 · ABAC v1	`yolo_pipeline/train_yolov8.ipynb`	2,826 images, 5 classes, 8 sessions, 3× augmentation	30 epochs, image size 640, batch 16, patience 15	mAP@50 0.764
4 · ABAC v2	`train_v2.ipynb`	6,585 images, 3 classes, 20 sessions, 2× augmentation	30 epochs, image size 1280, batch 8, patience 15	`best_v2_1280.pt`, 22 MB; mAP@50 0.883 on the test split

A later stage 3 run at image size 1280 produced `stage3_yolov8s_1280_ep30`, which is the starting point for stage 4. The sources give conflicting batch sizes for that run, so we state none here.

Stage 4 uses batch 8 because batch 16 at image size 1280 does not fit in T4 memory. The best epoch was about epoch 18. The final weights, `best_v2_1280.pt` (22 MB), are stored as `yolo_pipeline/best_v2_1280.pt`. The stage 4 training call:

```
MODEL_SIZE = 'yolov8s'
EPOCHS     = 30
IMAGE_SIZE = 1280
BATCH_SIZE = 8
model = YOLO('stage3_yolov8s_1280_ep30/weights/best.pt')
model.train(
    data=DATA_YAML, epochs=30, imgsz=1280, batch=8,
    patience=15, save=True, save_period=10,
    device=0, workers=2, exist_ok=True, plots=True,
)
```

For stage 2, we remapped the VisDrone classes by name onto three classes:

Table E.2 · How VisDrone classes were mapped for stage 2

VisDrone class	Used as
car, van	car
truck, bus	truck
motor, motorcycle, tricycle, awning-tricycle	motorcycle
pedestrian, people, bicycle	skipped

E.3 What each training setting means

Table E.3 · What each training setting means

Setting	What it means

Setting	What it means
`epochs` = 30	How many times the model sees the whole training set. Too few and it has not learned; too many and it starts to memorise the training images.
`imgsz` = 1280	The size in pixels that images are resized to before training. A larger size keeps small vehicles visible but needs more memory and time.
`batch` = 8	How many images go through the GPU at once before the weights are updated. They must all fit in GPU memory together, and 16 at image size 1280 does not fit on a T4.
`patience` = 15	Early stopping. If the validation mAP has not improved for 15 epochs in a row, training stops and keeps the best epoch, not the last one.
`save` = True	Write weight files to disk: `best.pt` (highest validation mAP) and `last.pt` (latest epoch).
`save_period` = 10	Also save a checkpoint every 10 epochs, so a Colab disconnect loses at most ten epochs.
`device` = 0	Which GPU to use. `0` is the first GPU, the T4 that Colab provides.
`workers` = 2	How many background processes load images while the GPU trains. Colab gives 2 CPU cores, so more would compete for them.
`exist_ok` = True	Write into an existing run folder instead of creating `run2`, `run3` and so on. Needed because the run folder is on Google Drive and we re-run cells.
`plots` = True	Produce the diagnostic images at the end: `results.png`, `confusion_matrix.png`, `F1_curve.png` and `PR_curve.png`. The confidence threshold is read from `F1_curve.png`.

E.4 Confidence threshold

The counter uses `CONF_THRESHOLD = 0.20`, set in `yolo_pipeline/detect_vehicles.py`. We read it off the F1 curve (`F1_curve.png`). The first model's curve peaked at 0.205. Step 7 of `train_v2.ipynb` shows the curve again, so the value can be read again for each new model. The F1 score is:

```
F1 = 2 x (precision x recall) / (precision + recall)
```

E.5 What was not recorded

We did not record the run dates or training times of stages 2 and 3, or the final VisDrone mAP of stage 2. We also cannot confirm that every run actually received a T4, because the device printouts were not saved. Stage 4's training time (4.07 hours) is in Section 3.6.

Appendix F · Development log

F.1 Counter failures caught by comparison with hand counts

Table F.1 · Counter failures caught by comparison with hand counts

Failure	Symptom	Cause
Spatial deduplication	counts roughly 30× too high	proximity is not identity

Failure	Symptom	Cause
Wall-clock windows	a 7 s window held 0.43 s of traffic	inference runs below real time
Hardcoded class table	one class silently dropped	model gained classes, indices shifted
Unmapped class name	**heavy = 0 while its mAP was 0.766**	see below
`--conf` never applied	threshold experiments were inert	constant read instead of argument

The fourth is the clearest. In the final calibration run the detector reported `car 229, motorcycle 71, heavy 0` for a window whose hand count was `car 189, motorcycle 72, heavy 37`. The model was not at fault; it had scored mAP@50 of 0.766 on heavy (Section 3.6). The table mapping class names to reporting groups had been written for the five-class taxonomy (`car`, `motorcycle`, `truck`, `bus`, `special type`) and never updated when Section 3.2 renamed the class `heavy`, and the lookup sent any unknown name to `"car"`.

The car count was 40 above the hand count, against a hand-counted heavy figure of 37, so the arithmetic showed the fault at once.

No training metric could reveal this, because the fault was in naming around the model, not in detection, and the output was well-formed JSON with believable totals. Only the hand-count comparison caught it, as it caught the other four. The counter now prints its full class mapping at startup and warns about any unmapped name:

```
[INFO] Classes  : car->car, heavy->heavy, motorcycle->moto
```

Appendix G · Platform drift measurement

G.1 The original estimator, and what replaced it

Every early drift figure came from `DriftTracker`, which has two faults pushing the same way. It chains frame-to-frame motion about 27,000 times per session, never rejecting a step (`0 estimates rejected` on every run), so the total wanders away from the truth and `max_shift` reports the largest wander. And it fits a translation plus similarity, which the real motion is not.

We re-measured by tracking fixed scenery landmarks with template matching (normalised cross-correlation of 80 × 80 patches), each matched on its own against frame 0, so no error chain builds up and one bad landmark cannot drag the others.

Table G.1 · Drift from the original estimator and as re-measured

Session	`DriftTracker`	Re-measured	Landmark agreement
`A_morning`	223 px	44-59 px	25-29%, not reliable

Session	DriftTracker	Re-measured	Landmark agreement
B_midday	205 px	52-80 px	42-53%
B_morning	164 px	47-92 px	70-87%
A_midday	107 px	45-53 px	88-89%
C_evening	56 px	9-13 px	93%

DriftTracker overstates drift two to five times on every session; real drift over 900 s is 9-92 px. A_morning cannot be measured this way, because masking out the road leaves grass, power lines and a barrier of repeating stripes, and template matching on a repeating pattern locks onto the wrong repeat. No drift figure should be quoted for it.

G.2 Why compensation failed: the model, not the tuning

Richer and richer motion models fitted to the landmarks show what kind of motion this is. Each figure is the median residual, the motion still unexplained after fitting.

Table G.2 · Residual motion left by each motion model

Session	motion	translation	similarity	homography
A_midday @900 s	45.3 px	10.6 px	3.6 px	**0.7 px**
B_morning @900 s	46.5 px	22.1 px	18.6 px	**1.8 px**
B_midday @900 s	91.4 px	41.2 px	46.8 px	**10.4 px**
C_evening @300 s	13.0 px	2.9 px	1.7 px	**1.2 px**

A homography explains 89-99% of the motion, because the camera's view of a flat road is changing; the similarity that drift.py assumes leaves 3-47 px, and a warp from the wrong model moves the polygon somewhere the road never was. Compensation moves the mean correction factor from 0.935× to 1.002× but grows the spread from 9.4% to 33.0%, **so the next session's factor cannot be predicted and cannot correct counts.**

The change in count also does not follow the drift it is meant to remove:

Table G.3 · Change in count with drift compensation, per session

Session	Re-measured drift	Change in cars + heavy
B_midday	80 px (largest)	+1.0%
B_morning	47 px	−5.5%
A_midday	45 px	−3.9%
A_morning	44 px (not reliable)	0.0%
C_evening	9 px (smallest)	−20.9%

The session that drifts least has the largest change in count. C_evening loses 38 of 182 vehicles while the tracker reports zero rejected estimates.

G.3 Where drift would matter, and how we reached the decision

A 50 px shift costs nothing near the camera (Section 4.6) but a great deal far from it, where these ROIs, drawn to the vanishing point, are only tens of pixels wide:

Table G.4 · Drift as a share of ROI width at the far edge

Session	Drift	ROI width at far edge	Drift as share of width
`B_midday`	80 px	55 px	145%
`B_morning`	47 px	33 px	143%
`A_midday`	45 px	90 px	50%
`A_morning`	44 px	90 px	49%
`C_evening`	9 px	52 px	17%

At the far edge the drone moves further than the road is wide; at the near edge the same drift is 6-26% of one lane. The problem sits almost entirely in the far third of the ROI, which Section 4.7 shows is unusable for counting anyway.

Two process notes. An early A/B test on `B_morning` alone moved the factor from 0.890× to 0.942×, a 5.5% change against a 3% threshold, and read as evidence for compensation; testing all five sessions reversed it, a confident answer from one sample in the wrong direction. And the original 56-223 px figure was never checked against an independent method because it looked believable, until the re-measurement in Appendix G.1 showed it two to five times too high.